PYTHON

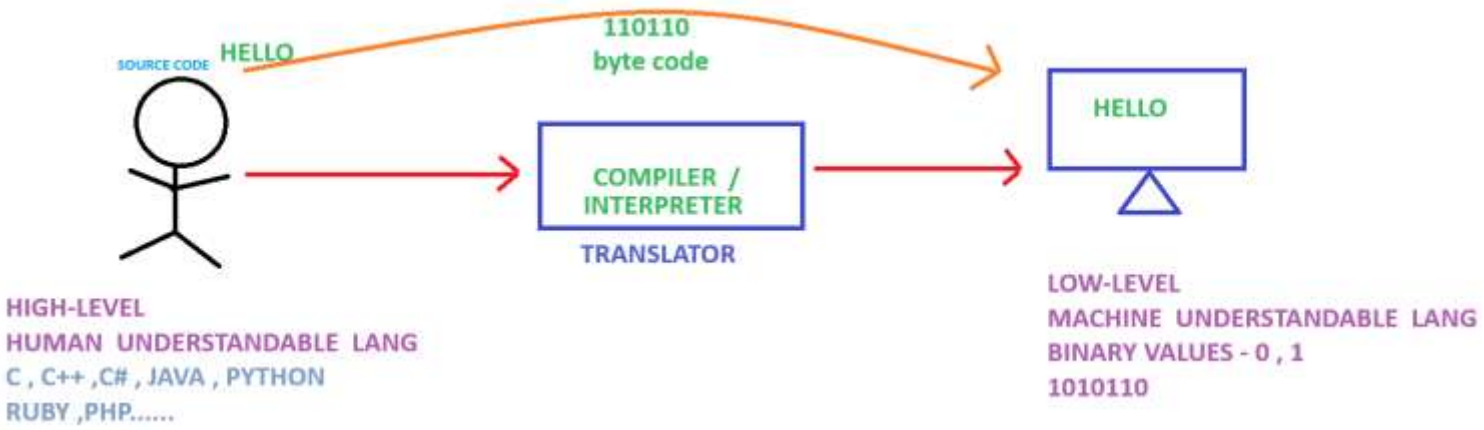
PROGRAMMING:

It is the process of creating a set of instruction that tells a computer how to perform a task.

SOFTWARE :

Collection of programs forms a software.

Computer Program Execution Process :



PYTHON:

Python is a high-level object-oriented programming language with object,modules,thread,exception..etc

It is simple but yet powerfull programing language

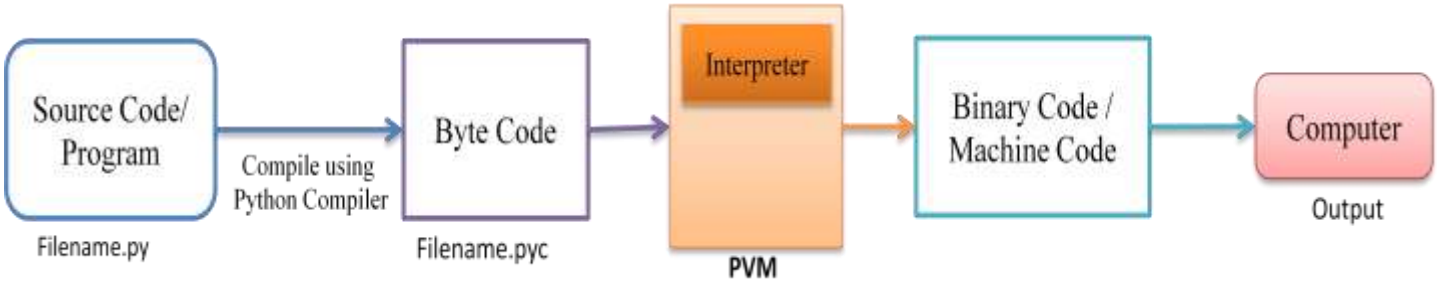
Python is a programming language that combines features of C and Java.

Application for Python

- Web Application - Django, Pyramid, Flask, Bottle
- Desktop GUI Application – Tkinter
- Console Based Application
- Games and 3D Application
- Mobile Application
- Scientific and Numeric
- Data Science
- Machine Learning - scikit-learn and TensorFlow
- Data Analysis - Matplotlib, Seaborn
- Business Application

Python Execution Process:

- Write Source Code / Program
- Compile the Program using Python Compiler
- Compiler Converts the Python Program into byte Code
- Computer/Machine Can not understand Byte Code so we convert it into Machine Code using PVM
- PVM uses an interpreter which understands the byte code and convert it into machine code
- Machine Code instructions are then executed by the processor and results are displayed



Python Virtual Machine

Python Virtual Machine (PVM) is a program which provides programming environment.

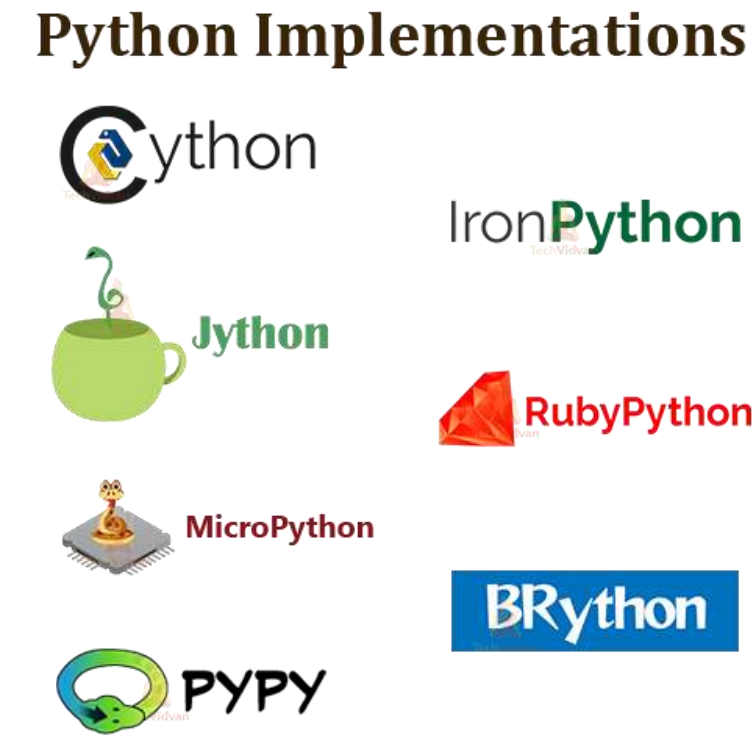
The role of PVM is to convert the byte code instructions into machine code so the computer can execute those machine code instructions and display the output.

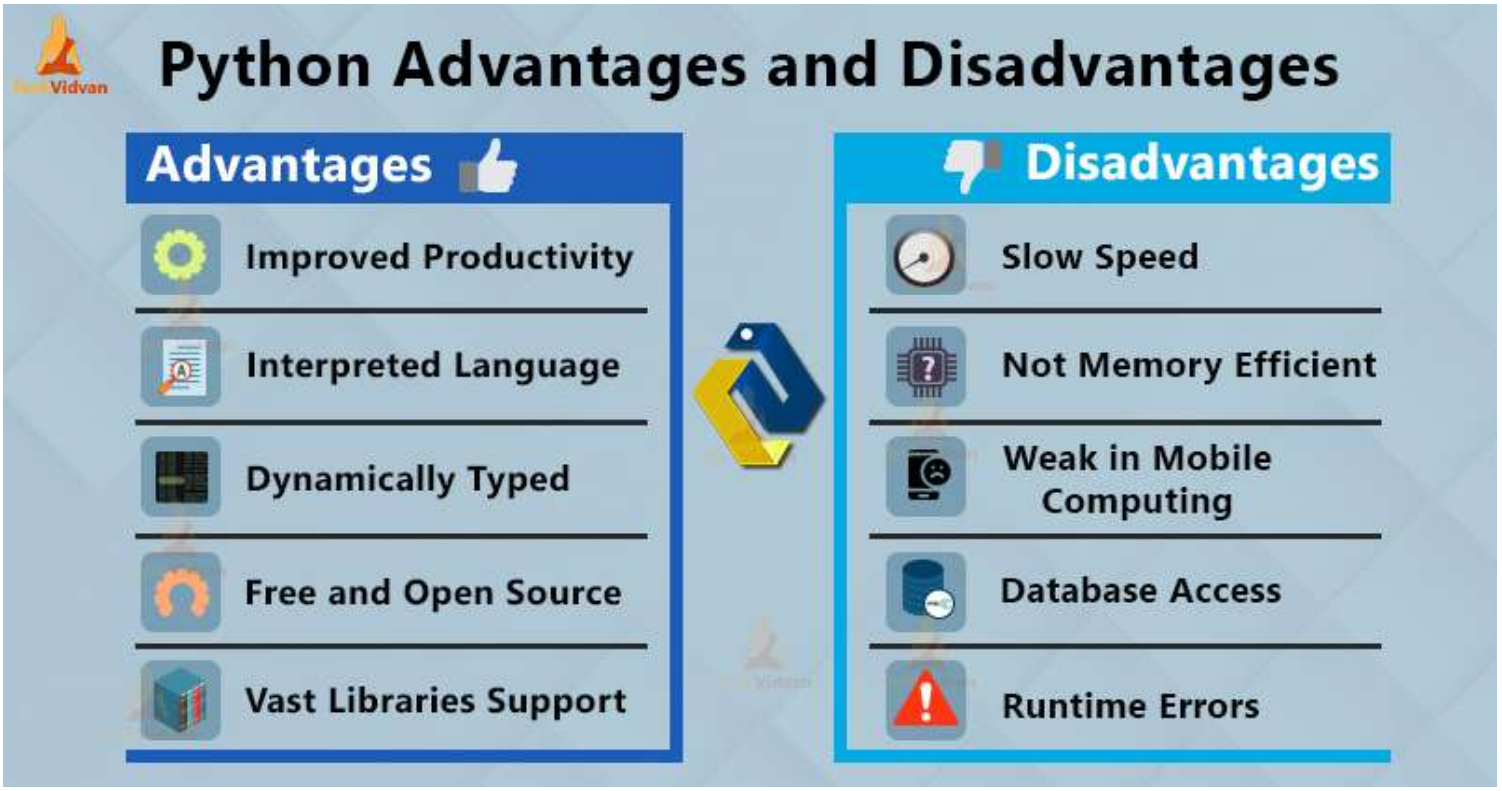
Interpreter converts the byte code into machine code and sends that machine code to the computer processor for execution.

Python Compiler –

A Python Compiler converts the program source code into byte code.

Type of Python Compilers :-





Executing Python Program

- Command Line Window

  Start -- python 3.12 - start typing

- Notepad or Notepad++

  Save file with .py

  Cmd prompt - goto file path - **python file_Name.py** -output

  python -m dis(disassembler)file_Name.py

  -----> to view bytecode inst

- PyCharm
- Visual Studio Code & python extension

Printing Statement :

```
print("Hello")
print("Hello \n world")
```

Comment

Comments are non-executable statements.

A Comment is used to describe the feature of a program. Comment helps to understand our program, not only ourselves but also other programmer.

There are two type of programs:-

- Single Line Comment
- Multi line Comment

Single Line Comment

These comments start with a hash symbol (#).

Ex:-

```
# I am single Line Comment
# This is my first Python Program
# Adding two numbers
```

Multi Line Comment

There is no concept of multi line comment in python but we can create string starting and ending with triple double quotes (”””) or triple single quotes (’’’) which can be used as block of comments.

Ex:-

```
"""                 '''
Comment Line 1      Comment Line 1
Comment Line 2      Comment Line 2
Comment Line 3      Comment Line 3

"""                 '''
```

Identifier

An identifier is a name having a few letters, numbers and special characters _ (underscore).

It should always start with a non-numeric character.

It is used to identify a variable, function, class etc.

Ex : -

X2
PI
Sigma
matadd
full_name

- **Python is case sensitive programming language.**
  d is not equal to D
  t is not equal to T
  rahul is not equal to Rahul
  rahul is not eqaul to RAHUL

Keywords or Reserved Words

Keywords means which contains some special meaning.

To view the keywords run this method – **help()** & type **keywords**

Python language uses the following keywords which are not available to users to use them as identifiers.

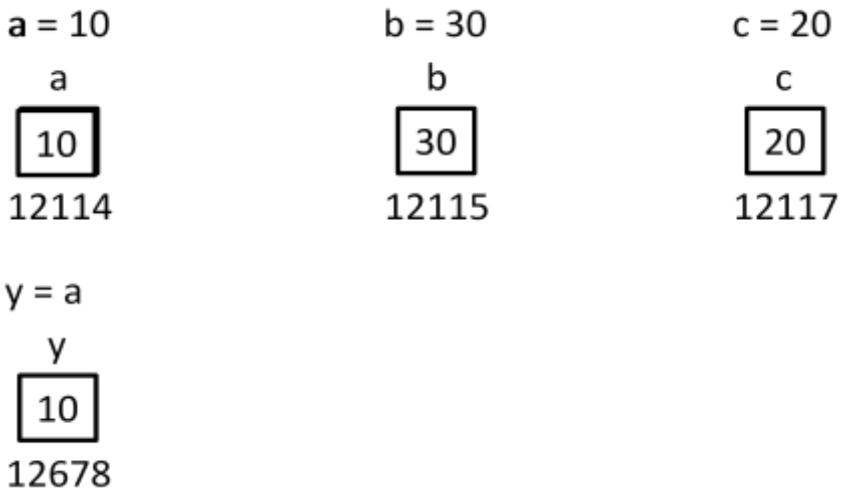
False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

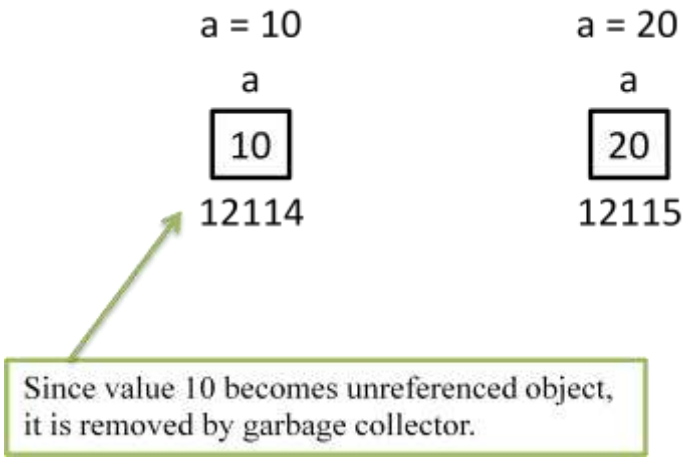
Variable

In C, Java or some other programming languages, a variable is an identifier or a name, connected to memory location.

In Python, a variable is considered as tag that is tied to some value.

Python considers value as objects.





Rules

- Every variable name should start with alphabets or underscore (_).
- No spaces are allowed in variable declaration.
- Except underscore (_) no other special symbol are allowed in the middle of the variable declaration
- A variable is written with a combination of letters, numbers and special characters _ (underscore)
- No Reserved keyword

Examples

Do	Don't
• A	and
• a	15name
• name	$city
• name15	Full$Name
• _city	Full Name
• Full_name	
• FullName	

```python
# Declaring and initializing Variable

# Individual assignment
a = 100
b = 10.223
c = "Students"
d = 'Students'

# Multiple assignment
e, f, g, h = 10, 10.23, "Students", 'Students'

# Multiple variable with same value
i = j = k = l = m = n = True

#variable reassignment
k=5
print(k) #5

k='hello'
print(k)  #hello

#variable concatination
first_name = "John"
last_name = "Smith"
full_name = first_name + " "+last_name
print(full_name) #John Smith

print(a)
print(b)
print(c)
print(d)
print(e)
print(f)
print(g)
print(h)
print(i)
print(j)
print(n)
print("$"*20)
```

Constants

A constant is an identifier whose value cannot be changed throughout the execution of a program whereas the variable value keeps on changing.

There are no constants in Python, the way they exist in C and Java.

In Python, It is not possible to define constant whose value can not be changed.

In Python, Constants are usually defined on a module level and written in all capital letters with underscores separating words but remember its value can be changed.

Ex:-

PI

TOTAL

MIN_VALUE

None –

None datatype represents an object that doesn't contain any value.

Print Statement

In Python, the print() function is used to display output on the screen.

It allows you to print text, variables, and expressions. Here are some examples of how to use the print() function:

1.Print text

```
print("Hello, World!")
```

2.Printing variables

```
name = "Alice"
age = 25
print("My name is", name, "and I am", age, "years old.")

print("My name is " + name + " and I am " + age + " years old.")
 #Error Str cannot concatinate with int

print("My name is " + name + " and I am " ,age , " years old.")
 #Use comma to overcome the abpve problem
```

3.Printing expressions

Functionality is available only from & above python 3.6

```
x = 10
y = 5
print("The sum of", x, "and", y, "is", x + y)
```

4.Printing Formatted Strings

```
name = "Bob"
age = 30
print(f"My name is {name} and I am {age} years old.")
```

5.Printing with Separators :

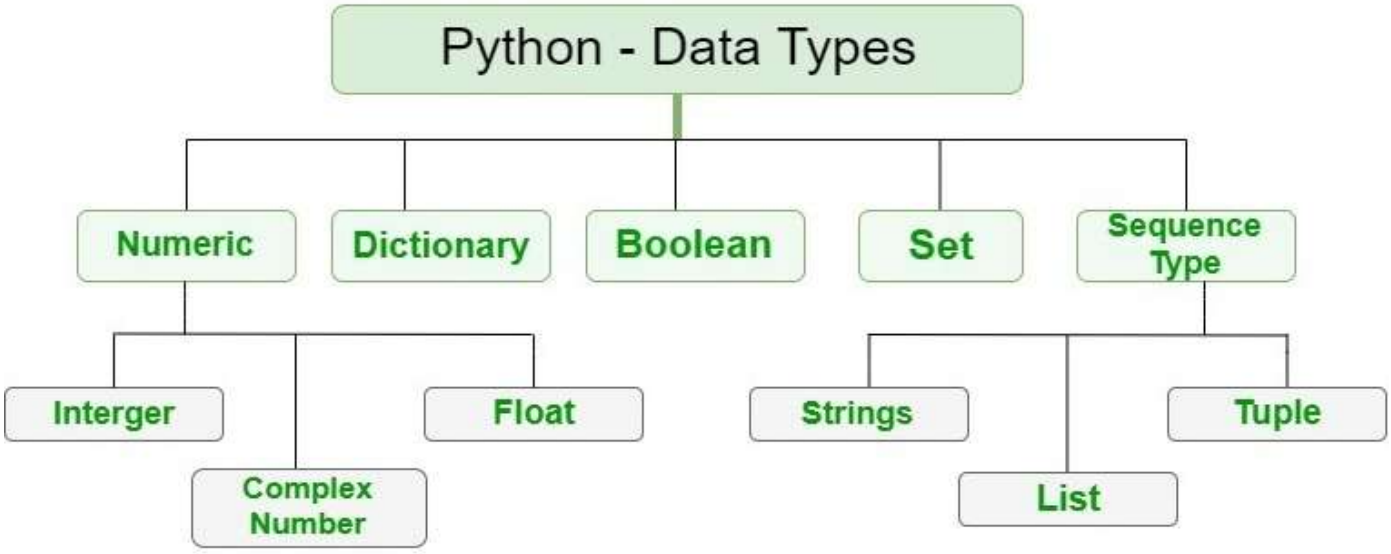
```
x = 1
y = 2
z = 3
print(x, y, z, sep="-")  # Output: 1-2-3
```

DataType

Datatype represents the type of data stored into a variable or memory.

Type of Data type :-

- Built-in Data type
- User Defined Data type --- Array , Class , Module



Int - 10 , 210 ,-5 ,-60

```
a=50;
print(a)
print(type(a))     # <class 'int'>
```

Float - 25.56, 10.5, -45.69, -0.8 , 5.1**e**5 = 5.1x10^5

Complex -

A complex number is a number that is written in the form of a + bj or a + bJ.

Where,

a = Real Part of the number

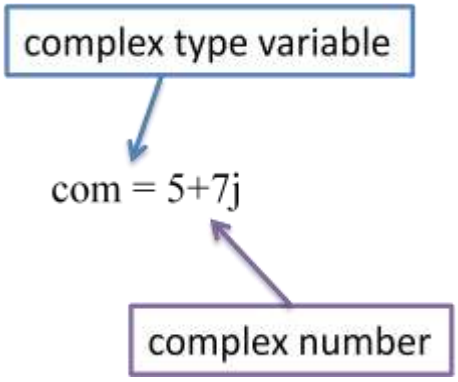
b = Imaginary part of the number

j or J = Square root value of -1

a and b may contain integer or float number.

Ex:- 5+7j , 0.8+2j

com = 5+7j



Boolean

The bool datatype represents boolean value True or False.
Python internally represents True as 1 and False as 0.

```
bool(True)
bool(False)

# all of the below evaluate to False. Everything else will evaluate to True in Python.
print(bool(None))     #false
print(bool({}))
print(bool(False))
print(bool(()))
print(bool(0))
print(bool(''))
print(bool(0.0))
print(bool(range(0)))
print(bool([]))
print(bool(set()))
```

Operations on Booleans

a. Arithmetic

It takes 0 for False, and 1 for True, and then applies the operator to them.

Addition

You can add two or more Booleans.

>>> True+False #1+0

Output

1

>>> True+True #1+1

Output

2

>>> False+True #0+1

Output

1

>>> False+False #0+0

Subtraction and Multiplication

The same strategy is adopted for subtraction and multiplication.

>>> False-True

Output

-1

Division

>>> False/True

Output

0.0

Remember that division results in a float.

>>> True/False

Output

Traceback (most recent call last):File "<pyshell#148>", line 1, in <module>
True/False

Modulus, Exponentiation, and Floor Division

The same rules apply for modulus, exponentiation, and floor division as well.

```
>>> False%True
>>> True**False
```

Output

1

```
>>> False**False
```

Output

1

```
>>> 0//1
```

Output

1

b. Relational

The relational operators we've learnt so far are >, <, >=, <=, !=, and ==.

All of these apply to Boolean values.

```
>>> False>True
```

Output

False

```
>>> False<=True
```

Output

True

Sequence Type

Following are sequence type:-

String

List

Tuple

Range

String –

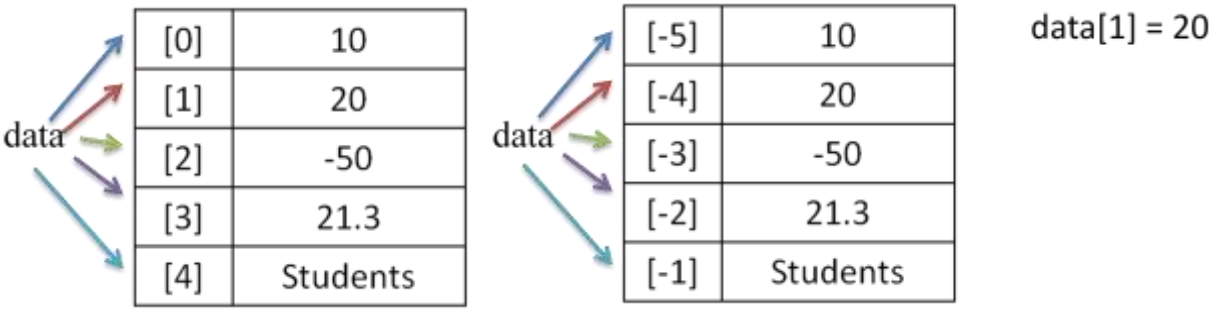
String represents group of characters.

Strings are enclosed in double quotes or single quotes.

Ex:- "Hello", "Students", 'Rahul'

List –

A list represents a group of elements.
A list can store different types of elements which can be modified**(immutable)**.
Lists are dynamic which means size is not fixed.
Lists are represented using square bracket **[ ]**.
Ex:- data = [10, 20, -50, 21.3, 'Students']



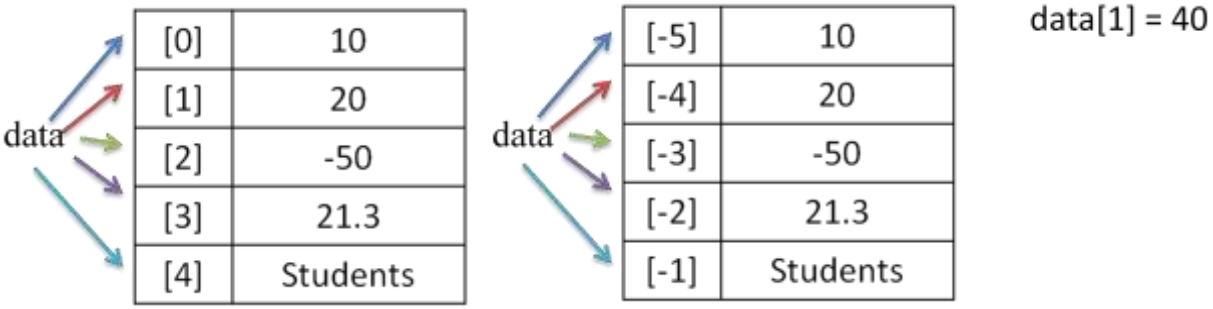
Tuple –

A tuple contains a group of elements which can be different types.

It is similar to List but Tuples are read-only which means we can not modify it's element **(immutable).**

Tuples are represented using parentheses **( ).**

Ex:- data = (10, 20, -50, 21.3, 'Students')

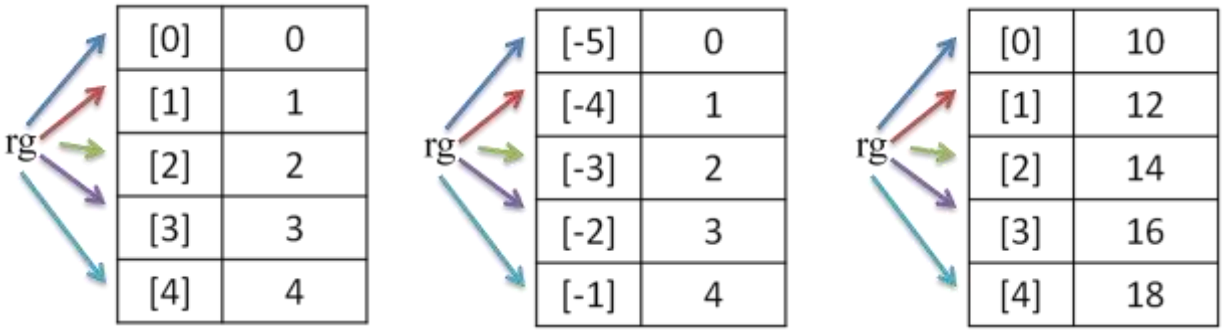


Range – Range represents a sequence of numbers.

The numbers in the range are not modifiable.

Ex:- rg = range(5) 0 1 2 3 4

rg = range(10, 20, 2) 10 12 14 16 18



Set Type

- A set is an unordered collection of elements much like a set in mathematics.
- The order of elements is not maintained in the sets. It means the elements may not appear in the same order as they are entered into the set.
- A set does not accept duplicate elements.
- Sets are unordered so we can not access its element using index.
- Sets are represented using curly brackets { }.

Ex:-

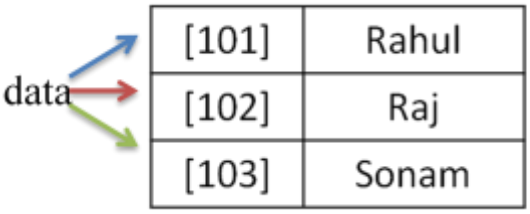
```
data = {10, 20, 30, "Students", "Raj", 40}
data = {10, 20, 30, "Students", "Raj", 40, 10, 20}
```

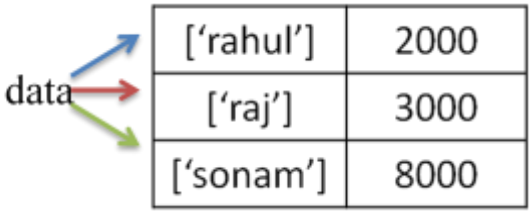
Dictionary/Mapping Type/ dict

A map represents a group of elements in the form of key value pairs.

Ex:-

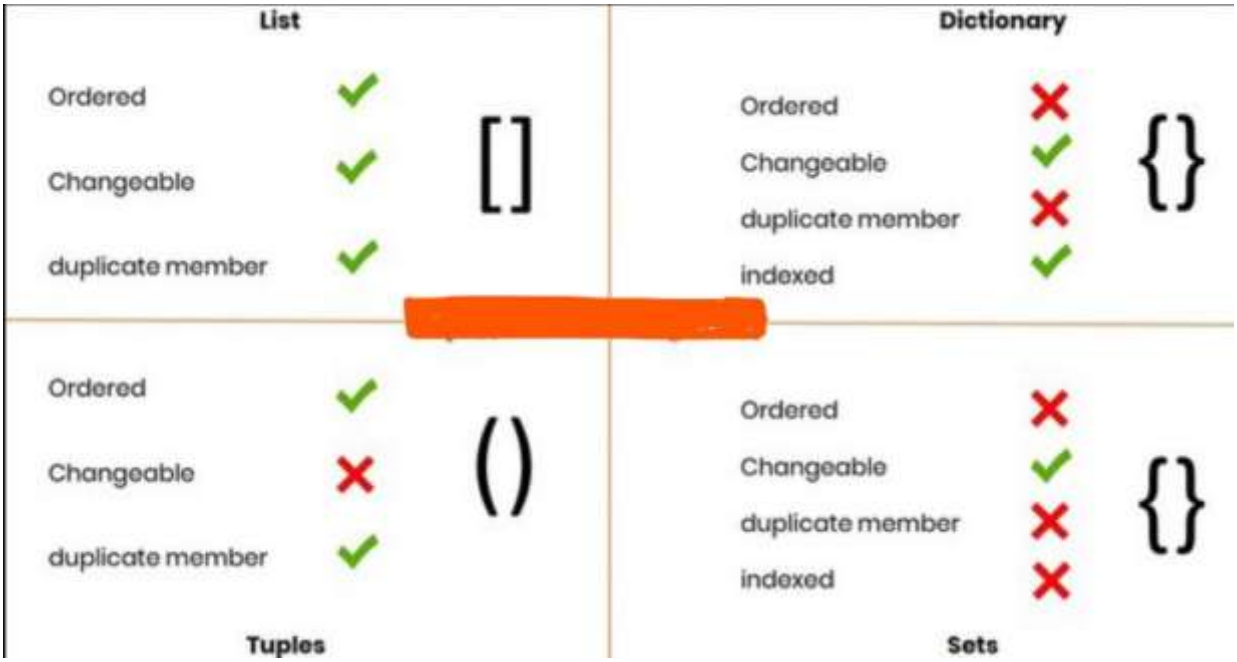
```
data = {101: 'Rahul', 102: 'Raj', 103: 'Sonam' }
data = {'rahul':2000, 'raj':3000, 'sonam':8000, }
```





Character

There is no concept of char data type in Python to represent individual character.


List
Ordered
Changeable
duplicate member
[]
Dictionary
Ordered
Changeable
duplicate member
indexed
{}
Ordered
Changeable
duplicate member
()
Tuples
Ordered
Changeable
duplicate member
indexed
{}
Sets

ASCII Values

ASCII (American Standard Code for Information Interchange) is a widely used character encoding standard that provides a way to represent characters as numeric codes.

ASCII uses 7 bits to represent a total if 128 characters, including uppercase and lowercase letters, digits, punctuation marks, control characters, and some special symbols.

Each character is assigned a unique numeric code from 0 to 127.

Here are some key ASCII values

- A to Z: ASCII values 65 to 90 represent uppercase letters.

- a to z: ASCII values 97 to 122 represent lowercase letters.

- 0 to 9: ASCII values 48 to 57 represent digits.

- Space: ASCII value 32 represents a space character.

- Special Characters:
Various special characters like punctuation marks, symbols, and control characters have their own ASCII values, such as:
    - ! (Exclamation Mark): ASCII value 33
    - @ (At Symbol): ASCII value 64
    - $ (Dollar Sign): ASCII value 36
    - % (Percent Sign): ASCII value 37
    - & (Ampersand): ASCII value 38
    - * (Asterisk): ASCII value 42
    - ...and many mire.

Obtain ASCII and Unicode in Python

In Python, you can obtain the Unicode code of a special character by using the built-in **ord()** function and the ASCII value of characters.

The ord() function accepts a string of unit length as an argument and returns the Unicode equivalence of the passed argument.

```
string = "€"
unicode_code = ord(string)

print("Unicode of",string,"=",unicode_code) #Unicode of € = 8364
```

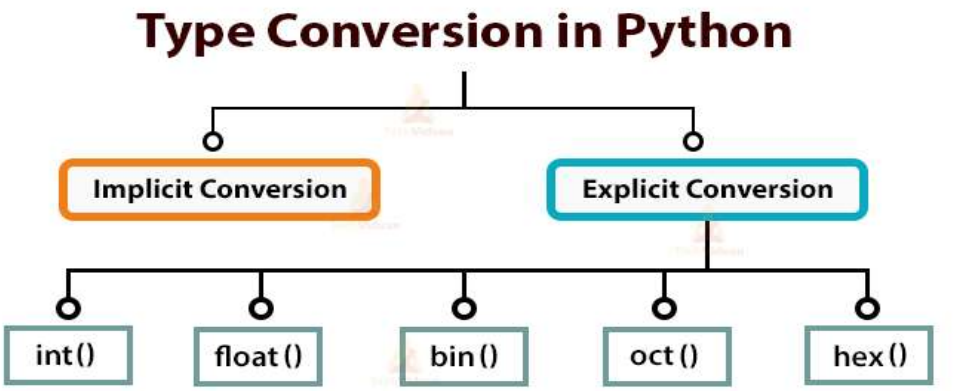
Conversely, Python **chr()** function is used to get a string representing of a character which points to a Unicode code integer. For example,

```
print(chr(97)) #a
```

This function takes an integer argument and throws an error if it exceeds from the specified range.

Type-Casting :

Converting one data type into another data type is called *Type Conversion*.



Implicit Type Conversion

In the Implicit type conversion, python automatically converts one data type into another data type.

Ex:-

1.Int to Float

```
a = 5
b = 2
value = a / b
print(value)          #2.5
print(type(value))    #<class 'float'>
```

2.Str +Str =Str

```
j = "Hello"
k = "Students"
p = j + k
print(p)          # HelloStudents
print(type(p))    #<class 'str'>

q = '20'
u = '10'
r = q + u
print(r)          #2010
print(type(r))    #<class 'str'>
```

3.Int + Str =Error

```
m = 20
n = 'Students'
t = m + n
print(t) #TypeError:unsupportedoperand type(s) for +:'int'a'& str'
```

Explicit Type Conversion

In the Cast/Explicit Type Conversion, Programmer converts one data type into another data type.

Function	Conversion
int()	string,float->int
float()	string,int->float
str()	int, float, list, tuple, dictionary -> string
list()	string, tuple, dictionary, set -> list
tuple()	string, list, set ->tuple
set()	string, list, tuple -> set
complex()	int, float -> complex

2.float to Int

```
float_num = 3.14
integer_num = int(float_num)
print(integer_num, type(integer_num))    #3 <class 'int'>
```

3.Str to Int

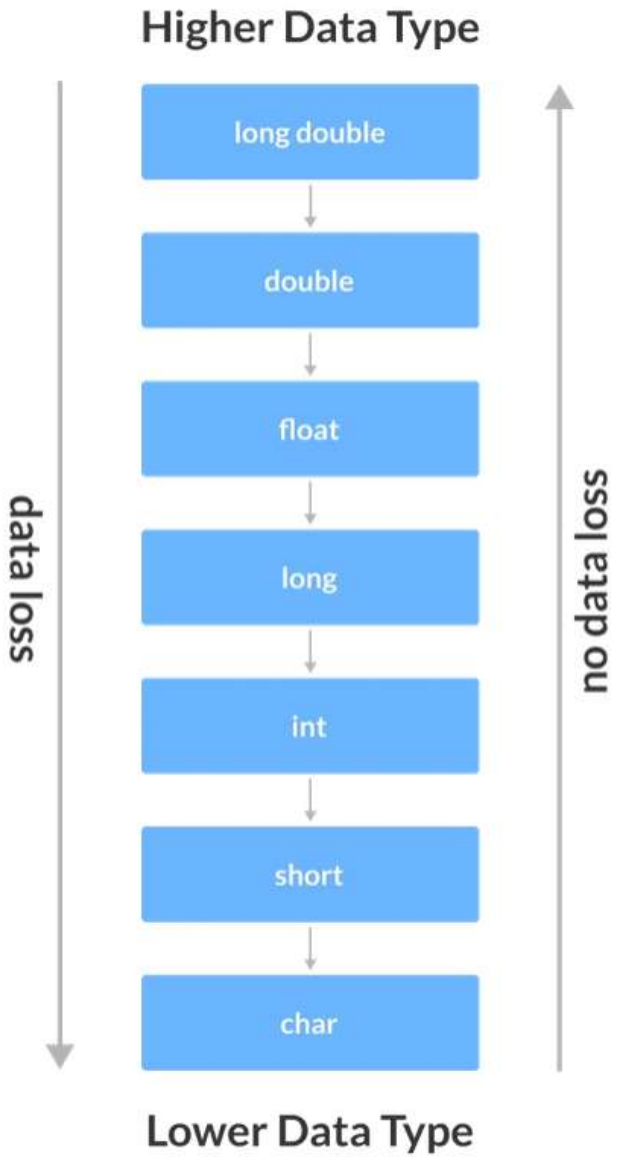
```
num_str = "123"
num = int(num_str)
print(num, type(num))    #123 <class 'int'>
```

```
# String to Integer
q = 20
u = '10'
print(type(u))  #<class 'str'>
```

```
r = q + int(u)
print(r)         #30
print(type(r))   #<class 'int'>
```

4.Int to Str

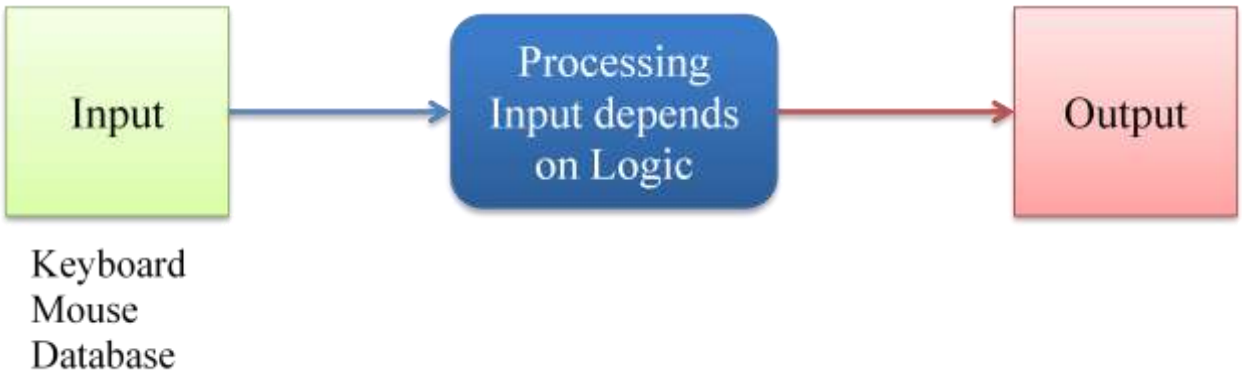
```
num = 42
num_str = str(num)
print(num_str, type(num_str)) #42 <class 'str'>


# Integer to Complex
n3 = 10
vn3 = complex(n3)
print(vn3)        #(10+0j)
print(type(vn3)) #<class 'complex'>


# Tuple to List
n5 = ("Rahul", "Ram", "Sonam", "Preet")
print(type(n5))      #<class 'list'>
vn5 = list(n5)
print(vn5)           #['Rahul', 'Ram', 'Sonam', 'Preet']
print(type(vn5))     #<class 'list'>
```



Input and Output

Input - The data given to the computer is called input.

Output – The results returned by the computer are called output.



Output Statements

print() Function –

The print() function is used to print the specified message to the output screen/device.

The message can be a string, or any other object.

Syntax:-

print(objects, sep='character', end='character' , flush=False)

sep - Separate the objects by given character. Character can be any string. Default is ' ' or can write none.

end – It indicates ending character for the line. Default is '\n' or can write none.

file - An object with a write method. Default is sys.stdout or can write none.

flush - A Boolean, specifying if the output is flushed (True) or buffered (False). Default is False

print("string" end='') –

When ending character is passed. It prints given character at the end. Default is '\n' or can write none.

Ex:-

```
print("Welcome", end='\n')       #Welcome

print("Welcome", end='')
print("to", end='')              #WelcometoStudents
print("Students")

print("Welcome", end='\t')
print("to", end='\t')
print("Students")                #Welcome to      Students
```

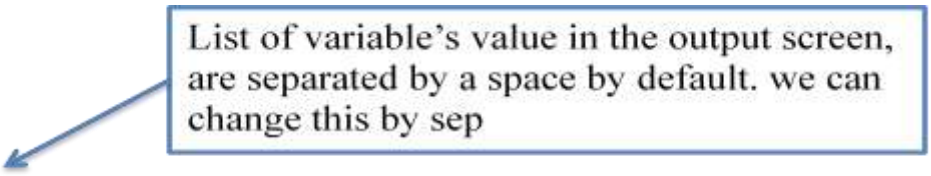
print(variable list)

- This is used to display the value of a variable or a list of variable.

Ex:-

```
1.a = 10
print(a)   #10
```

```
2.y = 30
x = 20
```




```
print(x, y)   #20 30
```

```
3.print(x, y, sep=',')     #20, 30
```

Input Statements

input( ) – This function is used to accept input from keyboard.

This function will stop the program flow until the user gives an input and end the input with the return key.

Whatever user gives as input, input function convert it into a string.

If user enters an integer value still input() function convert it into a string.

So if you need an integer you have to use type conversion.

Syntax:- input([prompt])

Prompt--is a string or message, representing a default message before input. It is optional

Ex:-

```
name   = input( )# getting user input without prompt

# Getting user input with prompt
name1  = input("Your Name: ")
mobile = input("Enter Your Mobile Number: ")


# Getting input and convert it into integer
mobile = input("Enter Mobile Number: ")
print(type(mobile))     #<class 'str'>

mb = int(mobile)
print(mb)
print(type(mb))     #<class 'int'>



# Getting input and convert it into integer directly
reg = int(input("Enter Registration Number: "))
print(reg)
```

Escape Sequence

Escape sequences – Escape sequences are control character used to move the cursor and print characters such as ', ". \ and so on.

Escape Sequence	Meaning
\a	Bell
\b	Backspace
\f	Formfeed
\n	NewLine
\r	Carriage Return
\t	Horizontal Tab
\v	Vertical Tab
\newline	Backslash and NewLine Ignored

Escape Sequence	Meaning
\\	Backslash
\'	Single Quote
\"	Double Quote

Ex-

```
print("Welcome to India")
print("Welcome to\nIndia")
print("Welcome to\tIndia")
print("Welcome to \"India\" ")
print("Welcome to \'India\' ")
print("Welcome \\ to \'India\' ")
print("Welcome to \b India ")
```

Ans:

```
Welcome to India

Welcome to

India

Welcome to India

Welcome to "India"

Welcome to 'India'

Welcome \ to 'India'

Welcome to India
```

Arithmetic Operator :

Operators	Meaning	Example	Result
+	Addition	4 + 2	6
-	Subtraction	4 - 2	2
*	Multiplication	4 * 2	8
/	Division	4 / 2	2
%	Modulus operator to get remainder in integer division	5 % 2	1
**	Exponent	5**2=5^2	25
//	Integer Division/ Floor Division	5//2 -5//2	2 -3

Relational/ Comparison Operators

Operators	Meaning	Example	Result
<	Less than	5<2	False
>	Greater than	5>2	True
<=	Less than or equal to	5<=2	False
>=	Greater than or equal to	5>=2	True
==	Equal to	5==2	False
!=	Not equal to	5!=2	True

Logical Operators

Operator	Meaning	Example	Result
and	Logical and	(5<2) and (5>3)	False
or	Logical or	(5<2) or (5>3)	True
not	Logical not	not (5<2)	True

Assignment Operators

Operator	Example	Equivalent Expression (m=15)	Result
=	y = a+b	y = 10 + 20	30
+=	m +=10	m = m+10	25
-=	m -=10	m = m-10	5
*=	m *=10	m = m*10	150
/=	m /=10	m = m/10	1.5
%=	m %=10	m = m%10	5
**=(exponent)	m**=2	m = m**2	225
//=(Divident)	m//=10	m = m//10	1

Bitwise Operators

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR / Bitwise XOR
~	Bitwise inversion (one's complement)
<<	Shifts the bits to left / Bitwise Left Shift
>>	Shifts the bits to right / Bitwise Right Shift

Membership Operators

The membership operators are useful to test for membership in a sequence such as string, lists, tuples and dictionaries.
There are two type of Membership operator:-

- in
- not in

in

This operators is used to find an element in the specified sequence.

It returns True if element is found in the specified sequence else it returns False.

Ex:-

```
st1 = "Welcome to India"
print("to" in st1)  #true

st2 = "Welcome top India"
print("to" in st2)  #true

st3 = "Welcome to India"
print("subs" in st3)    #false
```

not in

This operators works in reverse manner for in operator.
It returns True if element is not found in the specified sequence and if element is found, then it returns False.

Ex:-

```
st1 = "Welcome to India"
print("subs" not in st1)    #True

st2 = "Welcome to India"
print("to" not in st2)      #False

st3 = "Welcome top India"
print("to" not in st3)      #False
```

Identity Operators

The identity operators compare the memory locations of two objects.
Hence, it is possible to know whether two objects are same or not.
There are two types of Identity operator:-

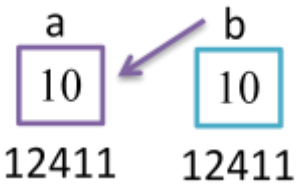
- is
- is not

is

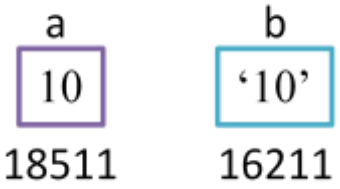
This operator is used to compare whether two objects are same or not.

It returns True if memory location of two objects are same else it returns False.

Ex:-




```
a = 10
b = 10
print(a is b)  #True
```

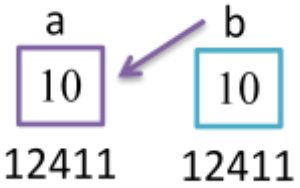



```
a = 10
b = '10'
print(a is b)   #False
```

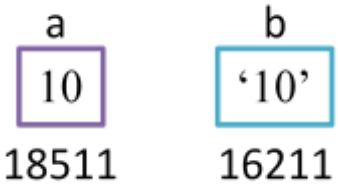
is not

This operator works in reverse manner for *is* operator.
It returns True if memory location of two objects are not same and if they are same it returns False.

Ex:-




```
a = 10
b = 10
print(a is not b)   #False
```




```
a = 10
b = '10'
print(a is not b)    #True
```

Operator Precedence and Associativity

- The computer scans an expression which contains the operators from left to right and performs only one operation at a time.
- The expression will be scanned many times to produce the result.
- The order in which various operations are performed is known as hierarchy of operations or operator precedence.

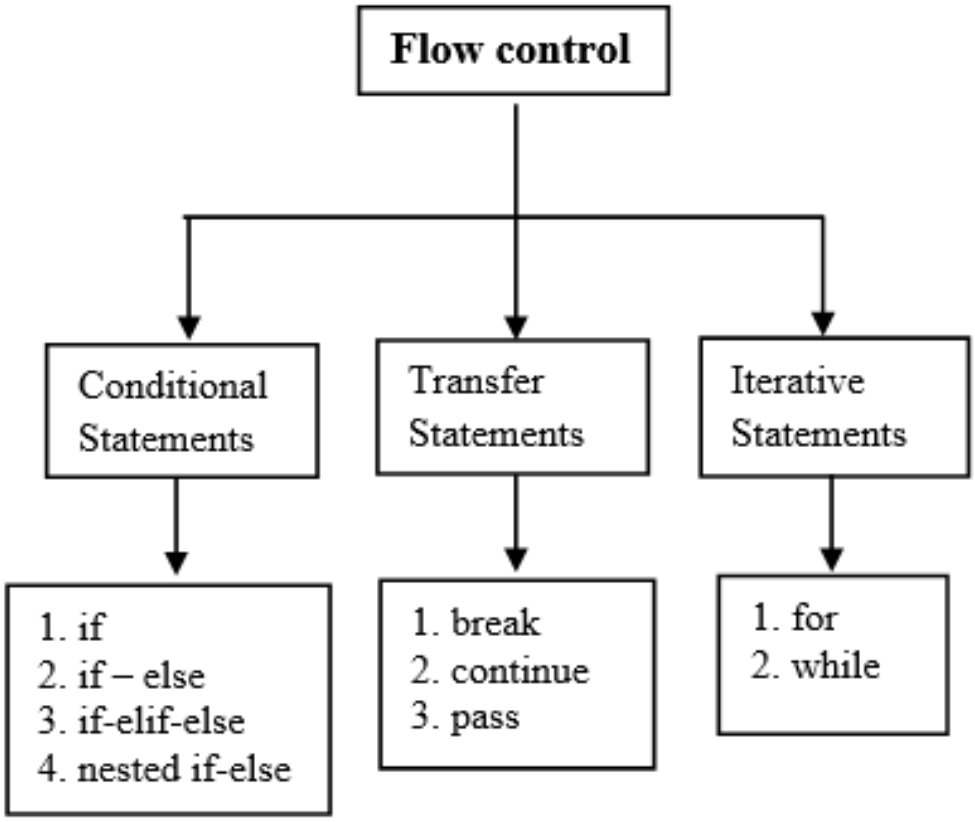
- Some of the operators of the same level of precedence are evaluated from left to right or right to left.
- This is referred to associativity.

value = (1+1)*24//3+4-1 ---ans ?**

Order of execution:

- Parentheses
- Exponentiation
- Multiplication, Division, Modulus and Floor Division
- Addition and Subtraction
- Assignment

```
value = (1+1)*2**4//3+4-1
    2*2**4//3+4-1
    2*16//3+4-1
    32//3+4-1
    10+4-1
    14-1
    13
```




```
#using if -- Calculate square
number = 6
if number > 5:
    print(number * number)
print('Next lines of code')


#using if else
password = input('Enter password ')
if password == "PYnative@#29":
    print("Correct password")
else:
    print("Incorrect Password")


#using nested if
num1 = int(input('Enter first number '))
num2 = int(input('Enter second number '))

if num1 >= num2:
    if num1 == num2:
        print(num1, 'and', num2, 'are equal')
    else:
        print(num1, 'is greater than', num2)
else:
    print(num1, 'is smaller than', num2)
```

Single statement suites

Whenever we write a block of code with multiple if statements, indentation plays an important role. But sometimes, there is a situation where the block contains only a single line statement.

```
number = 56
if number > 0: print("positive")
else: print("negative")
```

---

range( ) Function

range() function is used to generate a sequence of integers starting from 0 by default, and increments by 1 by default, till j-1.

Syntax:- range(start, stop, stepsize)

Start – Starting position.
If we do not mention start by default it's 0

*Stop – Ending position.
The range of integers stops one element prior to stop.
If stop is j then it will stop at exact j-1

Stepsize – Increment by stepsize.
If we do not mention start by default it's 1

Rules:-

- All argument must be integers, whether its positive or negative
- You can not pass a string or float number or any other type in a start, stop and stepsize.
- The stepsize value should not be zero.

Syntax:-

```
range( j )            0, 1, 2, 3, 4,……..., j-1

range(10)             0, 1, 2, 3, 4, 5, 6, 7, 8, 9

range(1, 10)          1, 2, 3, 4, 5, 6, 7, 8, 9

range(1, 10, 2)       1, 3, 5, 7, 9

range(-1, -10, -2)    -1  -3  -5  -7  -9

range(10, 0, -1)      10  9  8  7  6  5  4  3  2  1
```

for loop in Python

Using for loop, we can iterate any sequence or iterable variable. The sequence can be string, list, dictionary, set, or tuple.

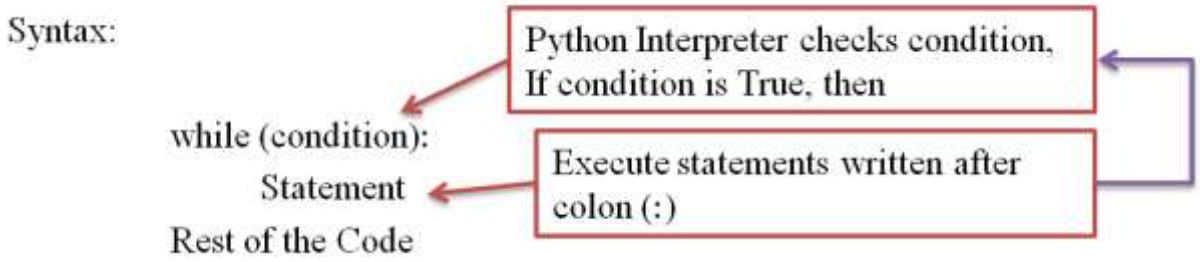
```
for item in [1,2,3,4]:
    print(item, end=",")

for i in range(1, 11):
    print(i)


x = range(5, 0, -1)
for n in x:
     print(n)


for i in range(-1, 5):
     print(i, end=", ") # prints: -1, 0, 1, 2, 3, 4,


for char in "Hello":
       print(char)


for i in [1, 2, 3, 4]:
  print(i, end=", ")           # 1, 2, 3, 4, The loop ended
else:
  print("The loop ended")
```

```
#Nested Loops
for num1 in range(3):
    for num2 in range(5, 8):
        print(num1, ",", num2)
```

While loop

In Python, The while loop statement repeatedly executes a code block while a particular condition is true.

In a while-loop, every time the condition is checked at the beginning of the loop, and if it is true, then the loop's body gets executed. When the condition became False, the controller comes out of the block.




```
a = 1
while(a<=10):
    print(a)
    a=a+1                          #1 to 10
print("Rest of the Code")
```

```
# Infinite While Loop
while True:
     print("Welcome")
print("Rest of the Code")
```

We **terminate** the loop by pressing **CTRL + C**

```
#Using else in while
num = 0
while( num<10 ):
    print(num)       #0 1 2 3 4
    num += 1
    if( num==5):
        break
else:
    print('Loop ended')



# Nested While Loop
i = 1
while i<=3:
     print("Outer Loop", i)
     i+=1
     j = 1
     while j<=5:
          print("Inner Loop", j)
          j+=1

print("Rest of Code")
```

Break Statement

It is used inside the loop to exit out of the loop.
It is useful when we want to terminate the loop as soon as the condition is fulfilled instead of doing the remaining iterations. It reduces execution time.
Whenever the controller encountered a break statement, it comes out of that loop immediately.

```
for num in range(10):
    if num > 5:
        print("stop processing.")
        break
    print(num)

# 0,1,2,3,4,5,stop processing.


num = 0
while( num <10 ):
    num +=1
    if(num==5): break
    print( num )

print("Loop ended")

#1,2,3,4,Loop ended
```

Continue statement

The continue statement is used to skip the current iteration and continue with the next iteration.

Continue statement is used in a loop to go back to the beginning of the loop.

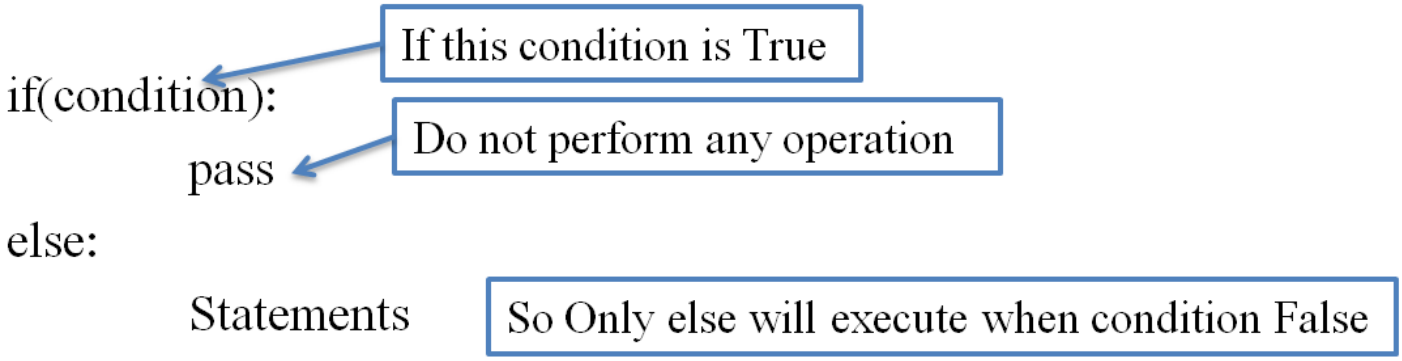
```
for num in range(3, 8):
    if num == 5:
        continue
    else:
        print(num,end=",") #3,4,6,7,


num = 0
while( num <10 ):
    num +=1
    if(num==5): continue
    print( num , end="," )

print("Loop ended")
#1,2,3,4,6,7,8,9,10,Loop ended
```

Pass statement

The pass is the keyword In Python, which won't do anything. Sometimes there is a situation in programming where we need to define a syntactically empty block.

We can define that block with the pass keyword.

A pass statement is a Python null statement. When the interpreter finds a pass statement in the program, it returns no operation. Nothing happens when the pass statement is executed.




```
months = ['January', 'June', 'March', 'April']
for mon in months:
    pass
print(months)
```

output: ['January', 'June', 'March', 'April']

String

String represents group of characters.
Strings are enclosed in double quotes or single quotes.
The *str* data type represents a String.

Ex:- “Hello”, “Students”, ‘Rahul’
str1 = “Students”

Creating String

1.Single Quotes
```
    str1 = 'Students'
```

2.Double quotes
```
    str2 = "Students"
```

3.Triple Single Quotes –
This is useful to create strings which span into several lines.

```
str3 = ''' Hello Guys
               Please Subscribe
               Students'''
```

4.Triple Double Quotes –
This is useful to create strings which span into several lines.

```
str4 = """Hello Guys
                Please Subscribe
                Students """
```

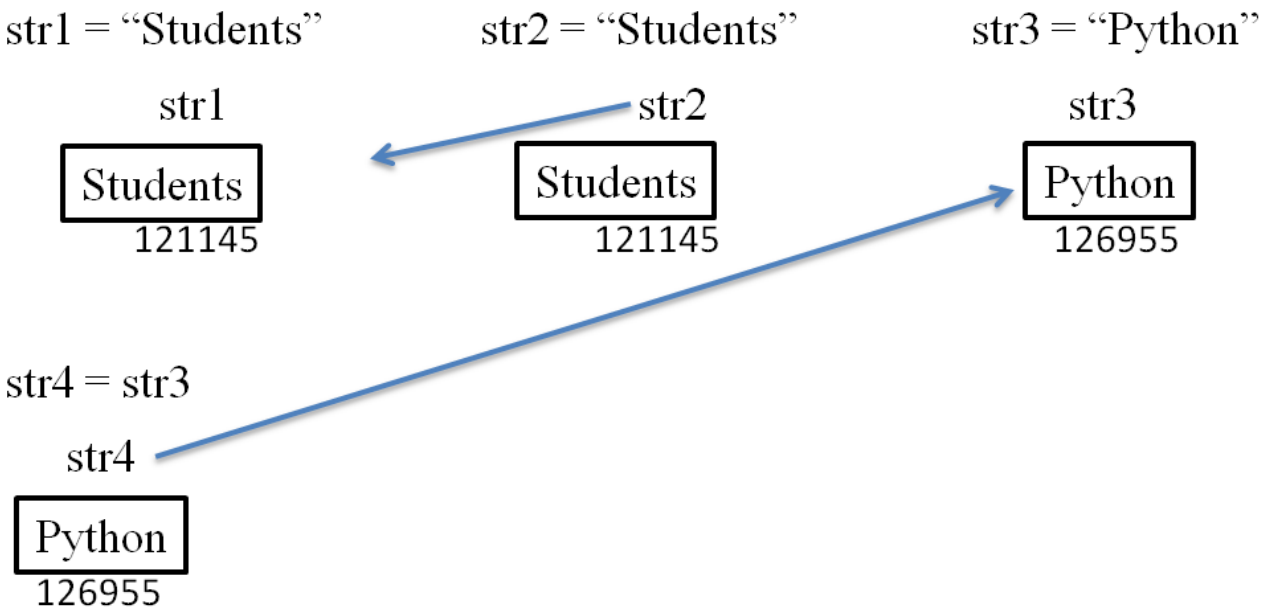
5.Double Quote inside Single Quotes
```
    str5 = 'Hello “Students” How are you'
```

6.Single Quote inside Double quotes
```
    str6 = "Hello 'Students' How are you"
```

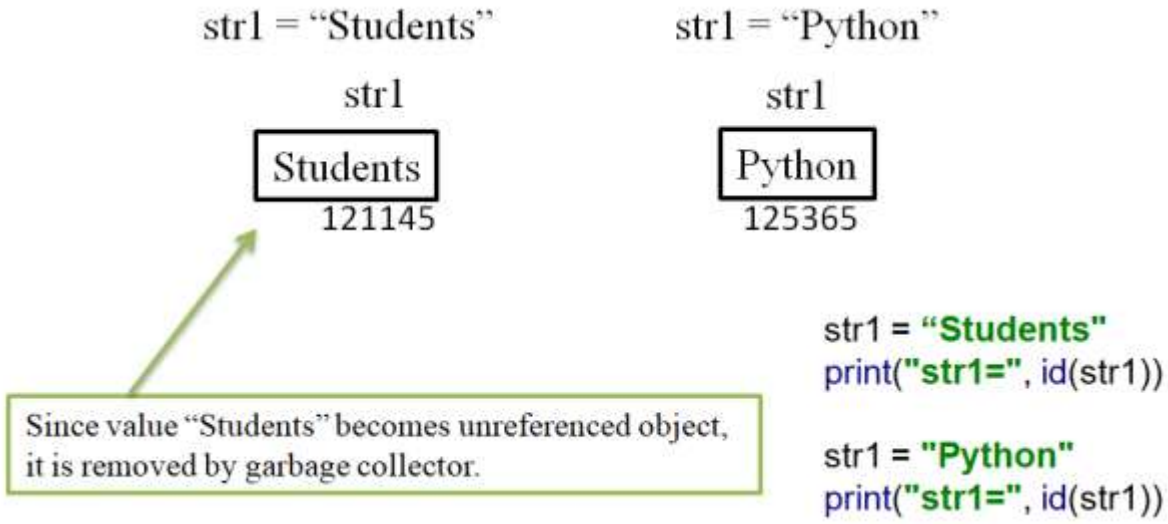
7.Using Escape Characters
```
    str7 = "Hello \nHow are You ?"
```

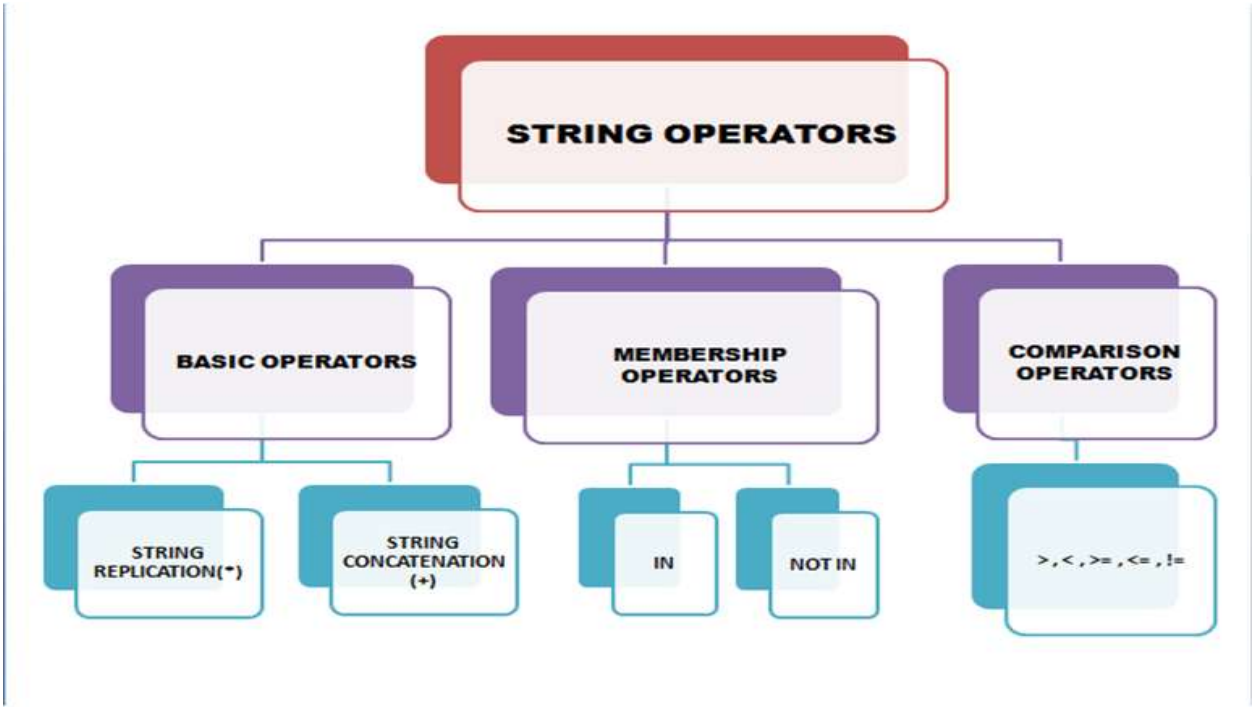
8.Raw String –
Row string is used to nullify the effect of escape characters.
```
str8 = r"Hello \nHow are You ?"       #Hello \nHow are You ?
```

Memory Allocation




```
str1 = "students"
str2 = "students"
str3 = "Python"
str4 = str3

print("str1=", str1, id(str1))      #str1= students 1611453319728
print("str2=", str2, id(str2))      #str2= students 1611453319728
print("str3=", str3, id(str3))      #str3= Python 140704264329296
print("str4=", str4, id(str4))      #str4= Python 140704264329296
```

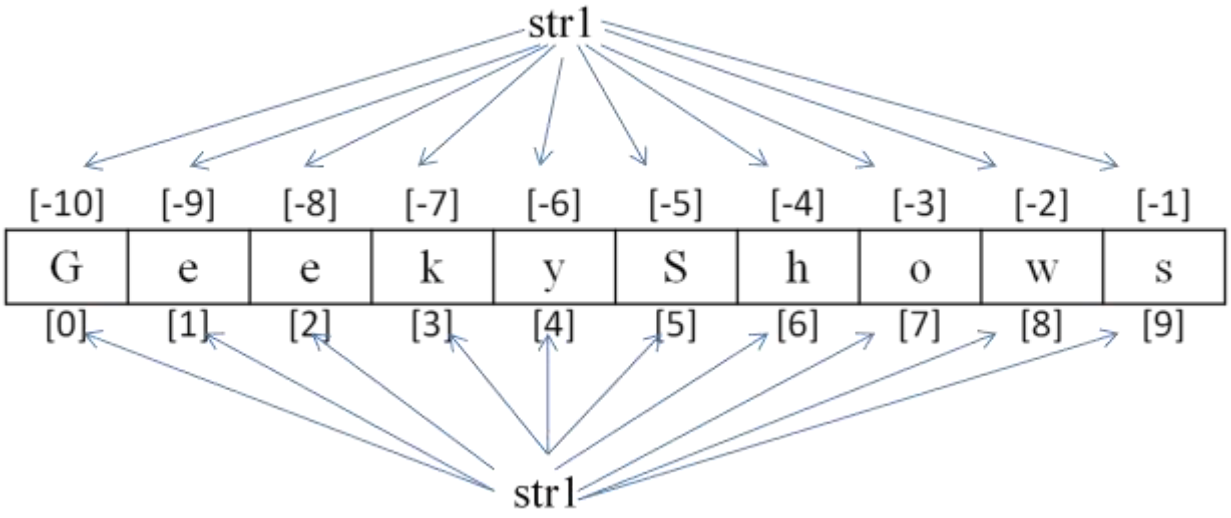




Index

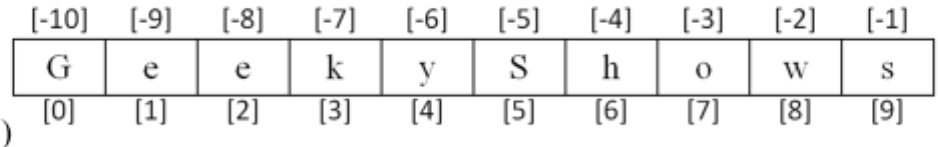
Index represents the position number of characters in a string.

Ex:- str1 = "Students"



Accessing Character and String

str1 = “Students”




```
print(str1[0])
print(str1[1])
print(str1[2])
print(str1[3])
print(str1[-1])

print(str1)
```

String Length

Length of string represents the number of characters in a string.
len( ) Function is used to get length of string.
Ex:-

```
str1 = "Students"
n = len(str1)
Print(n)      #8
```

Access using Loop

```
str1 = "Students"
for c in str1:                  #AccessUsingForLoopWithoutIndex
       print(c)

for i range(len(str1)):         #AccessUsingForLoopWithIndex
       print(str1[i])

i = 0
while i < len(str1) :           #AccessUsingWhileLoopWithIndex
       print(str1[i])
       i+=1
```

Mutable and Immutable Object

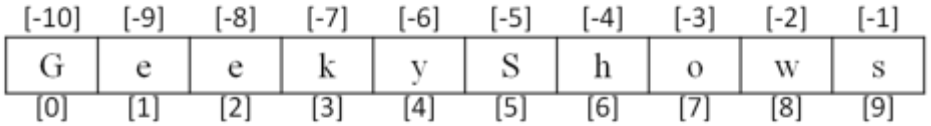
- Mutable Object – Mutable objects are those object whose value or content can be changed as and when required.
  Ex:- List, Set, Dictionaries

- Immutable Object – Immutable objects are those object whose value or content can not be changed.
  Ex:- Numbers, String, Tuple

Immutable String

In Python, Strings are immutable object which means it's value or content can not be changed.

str1 = "Students"



```
# It will show TypeError, you can not modify String
str1 = "Students"
str1[4] = "i"
for c in str1:
     print(c)
```

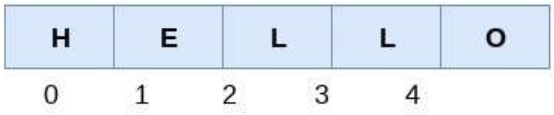
Slicing String

Slicing on String can be used to retrieve a piece of the string.
Slicing is useful to retrieve a range of elements.
Syntax:-

```
new_string_name = string_name[start:stop:stepsize]
```

- **start** – It represents starting point. By default its 0
- **stop** – It represents ending point.
- **stepsize** – It represents step interval. By default It is 1
- If start and stop are not specified then slicing is done from 0^{th} to $n-1^{th}$ elements.
- Start and Stop can be negative number.

str = "HELLO"



str[0] = 'H'

str[1] = 'E'

str[2] = 'L'

str[3] = 'L'

str[4] = 'O'

str[:] = 'HELLO'

str[0:] = 'HELLO'

str[:5] = 'HELLO'

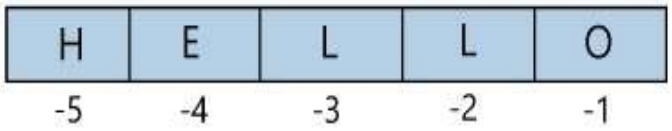
str[:3] = 'HEL'

str[0:2] = 'HE'

str[1:4] = 'ELL'

---

str = “HELLO”



str[-1] = ‘O’

str[-2] = ‘L’

str[-3] = ‘L’

str[-4] = ‘E’

str[-5] = ‘H’

str[-3:-1] = ‘LL’

str[-4:-1] = ‘ELL’

str[-5:-3] = ‘HE’

str[-4:] = ‘ELLO’

str[::-1] = ‘OLLEH’

Repetition Operator

Repetition operator is used to repeat the string for several times.
It is denoted by *

Ex:-

```
1. "$" * 7

2.str1 = "Students"
  print(str1*5)

3.print(str1[0:4]* 3)
```

Concatenation Operator

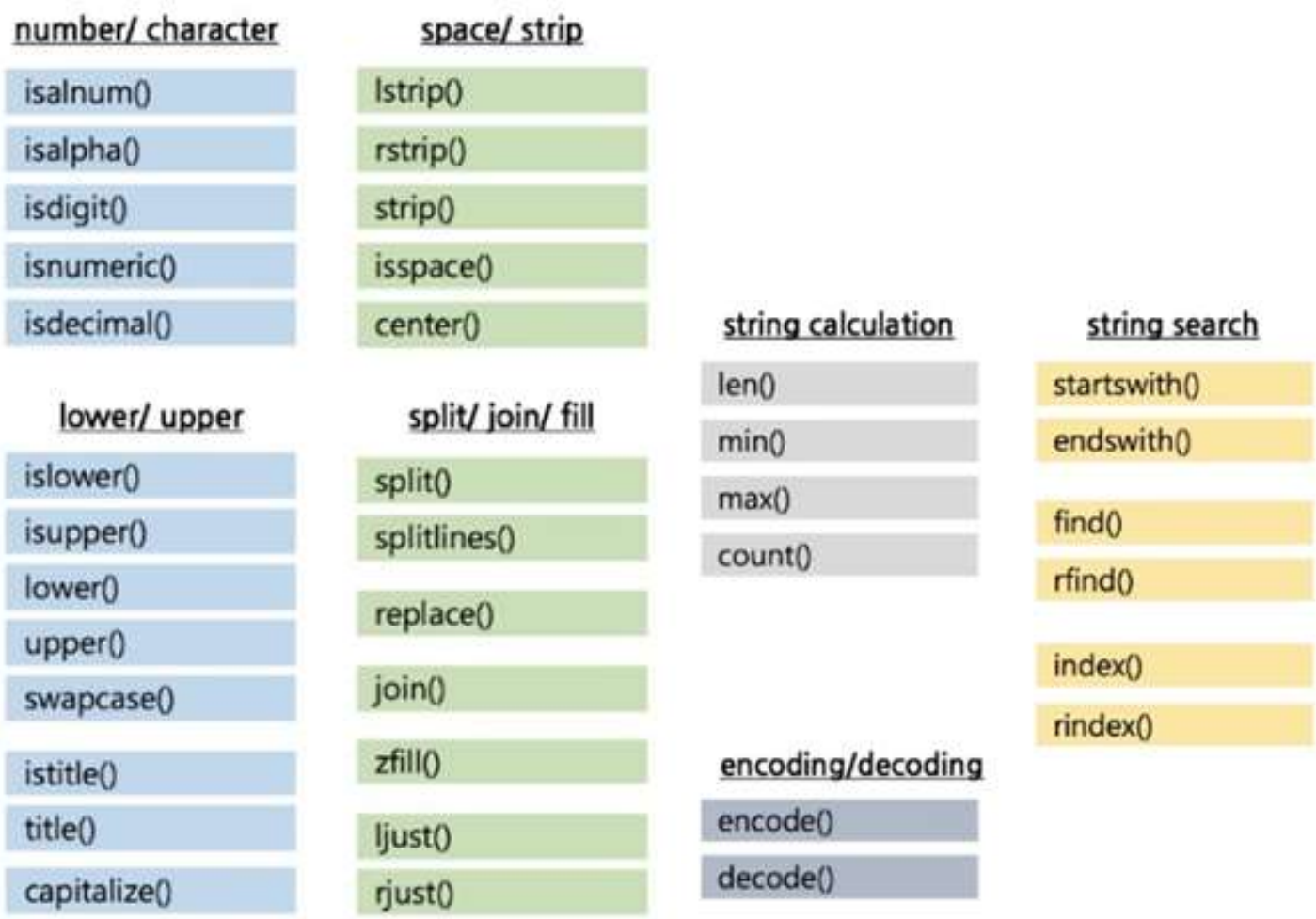
Concatenation operator is used to join two string.
It is denoted by +

Ex:- “Hi” + “Students”

```
a = "Hello“
b = "World“
c = a + b
print(c)
```

```
str1                    =                    “Students"
str2                    =                    “Students"
str3                     =                     "Python"
str4                        =                        "A"
str5 = "B“

result1          =          str1          ==          str2
result2          =          str1          ==          str3
result3 = str4 < str5
print(result1)                                      #True
print(result2)   #False
print(result3)   #True
```

In-built Strings :



Function

What are Functions in Python?

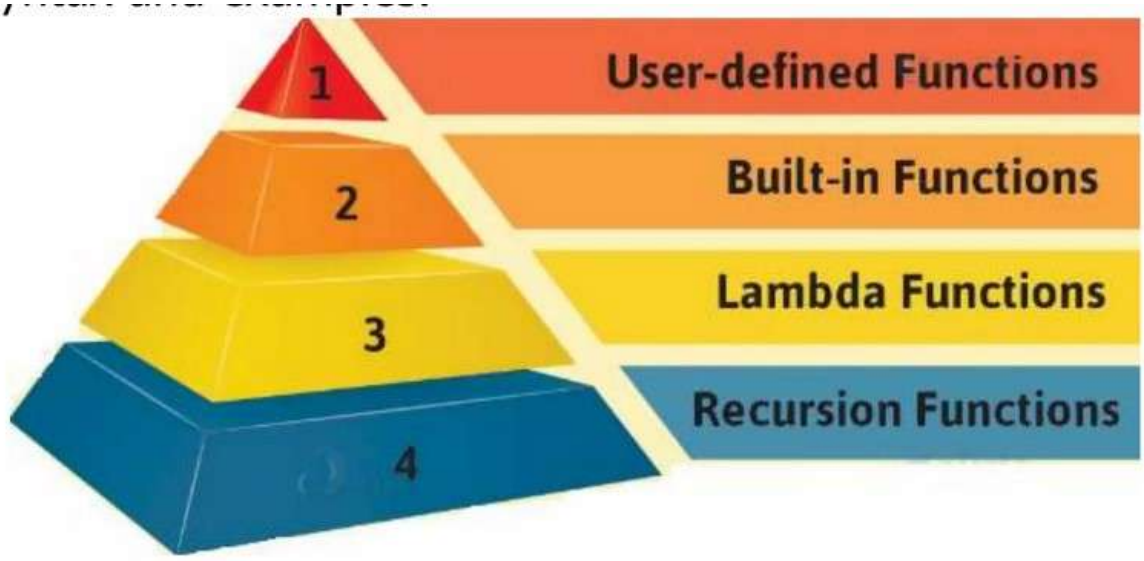
A function is a **block of code** that has a **name** and you can **call** it. Instead of **writing** something 100 times, you can **create a function** and then **call it 100 times.**

You can call it **anywhere** and **anytime** in your program.

This adds **reusability** and **modularity** to your code.

Functions can take **arguments** and **return values.**

Type of Functions:-



Advantage of Function

- Write once and use it as many time as you need. This provides code reusability.
- Function facilitates ease of code maintenance.
- Divide Large task into many small task so it will help you to debug code
- You can remove or add new feature to a function anytime.

Built-in Functions

A	E	L	R
abs()	enumerate()	len()	range()
all()	eval()	list()	repr()
any()	exec()	locals()	reversed()
ascii()			round()
	F	M	
B	filter()	map()	S
bin()	float()	max()	set()
bool()	format()	memoryview()	setattr()
breakpoint()	frozenset()	min()	slice()
bytearray()			sorted()
bytes()	G	N	staticmethod()
	getattr()	next()	str()
C	globals()		sum()
callable()		O	super()
chr()	H	object()	
classmethod()	hasattr()	oct()	T
compile()	hash()	open()	tuple()
complex()	help()	ord()	type()
	hex()		
D		P	V
delattr()	I	pow()	vars()
dict()	id()	print()	
dir()	input()	property()	Z
divmod()	int()		zip()
	isinstance()		
	issubclass()		_
	iter()		__import__()

- We can define a function using **def** keyword followed by function name with parentheses.
- This is also called as Creating a Function, Writing a Function, Defining a Function.
- A Function runs only when we call it, function can not run on its own.

Syntax : -

```
def Function_name ():
        Local Variable
        block of statement
        return (variable or expression)
```

Function Body

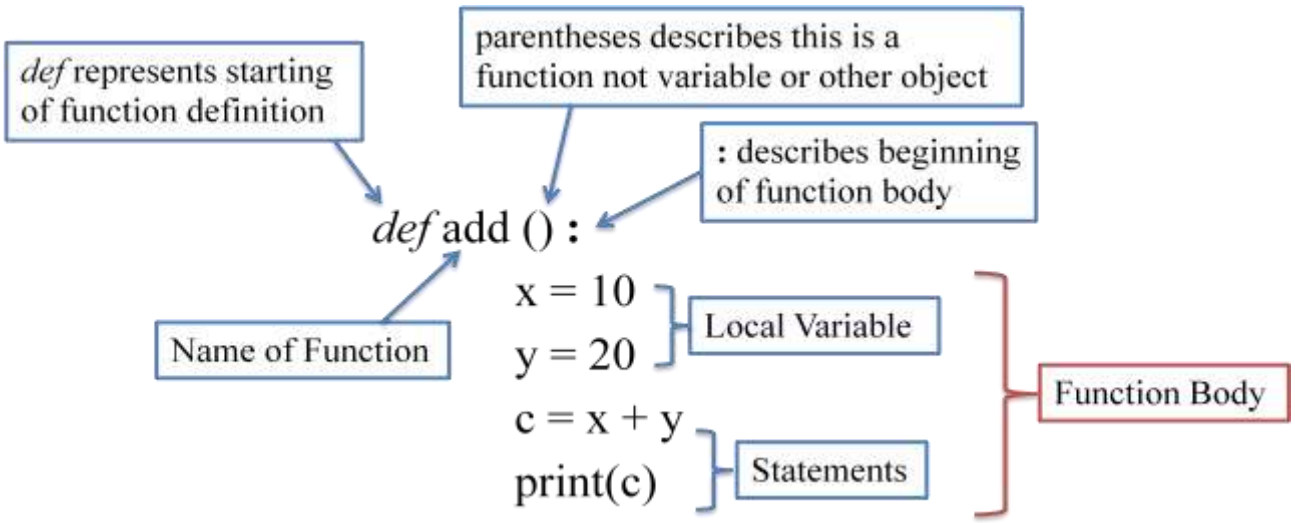
Syntax : -

```
def Function_name (para1, para2, ...):
        Local Variable
        block of statement
        return (variable or expression)
```

Function Body

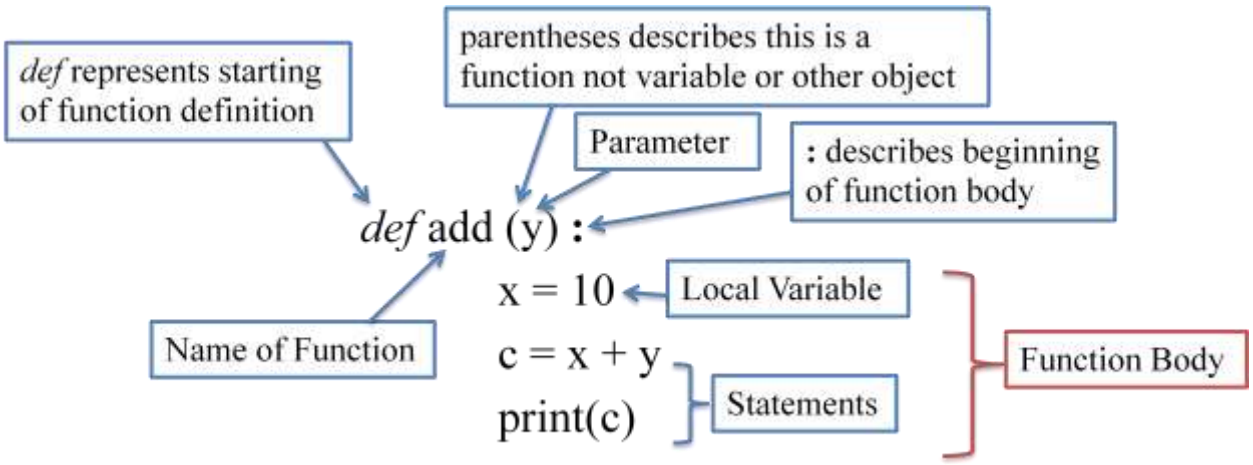
Note – Need to maintain proper indentation

Zero Parameterized Function :




```
add()
```

Parameterized Function :




```
add(20)
```

Return Statement

Return statements can be used to return something from the function.
In Python, it is possible to return one or more variables/values.

Syntax : - return (variable or expression);

Ex: -

```
def add (y) :
        x = 10
        c = x + y
        return c
sum = add (20)
print(sum)
```

```
def add (y) :
        x = 10
        return x + y
sum = add (20)
print(sum)
```

```
def add_sub (y) :
        x = 10
        c = x + y
        d = y - x
        return c, d
sum, sub = add (20)
print(sum)
print(sub)
```

Nested Function

When we define one function inside another function, it is known as Nested Function or Function Nesting.

Ex:-

```
def disp():
      def show():
          print("Show Function")
      print("Disp Function")
      show()

disp( )
```

Pass a Function as Parameter

We can pass a function as parameter to another function.

Ex:-

```
def disp(sh):
     print("Disp Function" + sh())

def show():
     return " Show Function"

disp(show)
```

Function return another Function

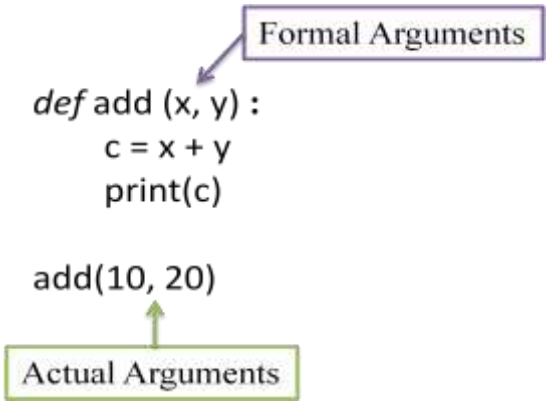
A function can return another function.

Ex:-

```
def disp():
        def show():
             return "Show Function"
        print("Disp Function")
        return show

r_sh = disp()
print(r_sh())
```

Actual and Formal Argument

- Formal Argument - Function definition parameters are called as formal arguments
- Actual Argument - Function call arguments are actual arguments



Type of Actual Arguments

- Positional Arguments
- Keyword Arguments
- Default Arguments
- Variable Length Arguments
- Keyword Variable Length Arguments

1. Positional Arguments

```
def pw (x, y) :
        z = x**y
        print(z)

pw(5, 2)
```

2. Keyword Arguments

```
def show (name, age) :
        print(name, age)

show(name="Kumar" , age=62)
```

3. Default Arguments

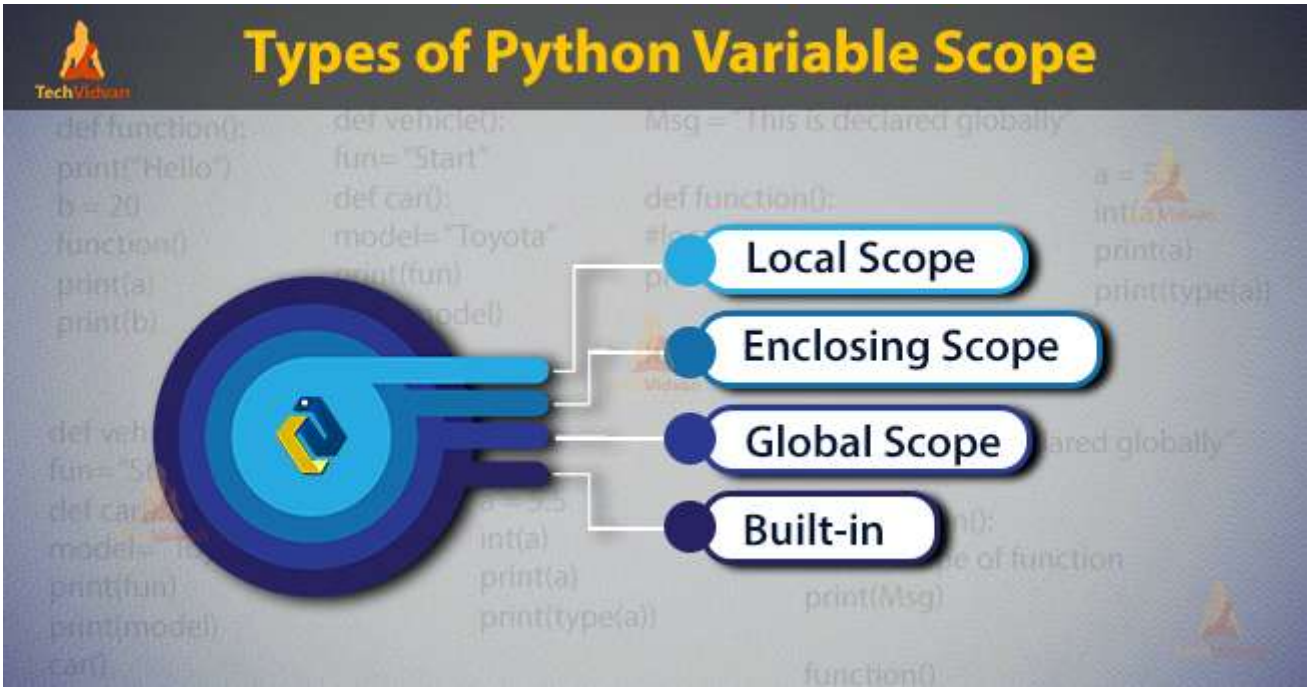
```
def show (name, age=27) :
        print(name, age)

show(name="Raam", age=62)
```

4. Variable Length Arguments

```
def add (*num) :
        z = num[0]+num[1]+num[2]
        print(z)

add(5, 2, 4)
```

5. Keyword Variable Length Arguments

```
def add (**num) :
        z = num['a']+num['b']+num['c']
        print(z)

add(a=5, b=2, c=4)
```



Local Variables

The variable which are declared inside a function called as Local Variable.

Local variable scope is limited only to that function where it is created.

It means local variable value is available only in that function not outside of that function.

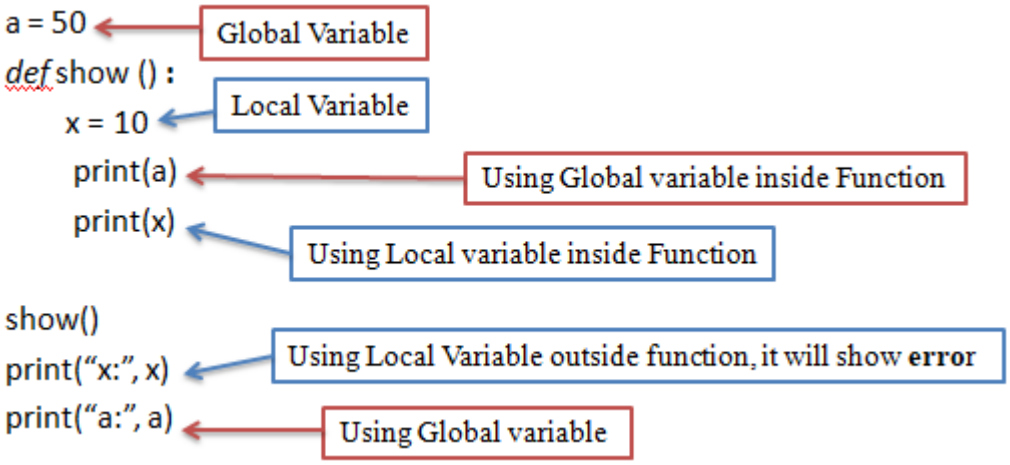
```
def add (y) :
    x = 10          <- Local Variable
    print(x)
    print(x + y)    <- Using Local variable inside Function
add(20)
print(x)            <- Using Local Variable outside function, it will show error
```

Global Variables

When a variable is declared above a function, it becomes global variable.

These variables are available to all the function which are written after it.

The scope of global variable is the entire program body written below it.



Global Keyword

If local variable and global variable has same name then the function by default refers to the local variable and ignores the global variable.

It means global variable is not accessible inside the function but possible to access outside of function.

In this situation, If we need to access global variable inside the function we can access it using global keyword followed by variable name.

```
a = 50
def show () :
        a = 10
        print(a)
show()
print("a:", a)
```

```
a = 50
def show () :
        global a
        print(a)    #50
        a = 20
        print(a)    #20
show()
print("a:", a)      #20
```

Enclosing Scope

A scope that isn't **local** or **global** comes under enclosing scope.

```
def vehicle():
  fun= "Start"
  def car():
    model= "Toyota"
    print(fun)
    print(model)
  car()
vehicle()
```

Output:

```
Start
Toyota
```

Anonymous Function or Lambdas

A function without a name is called as Anonymous Function.

It is also known as Lambda Function.

Anonymous Function are **not** defined using *def* keyword rather they are defined using *lambda* keyword.

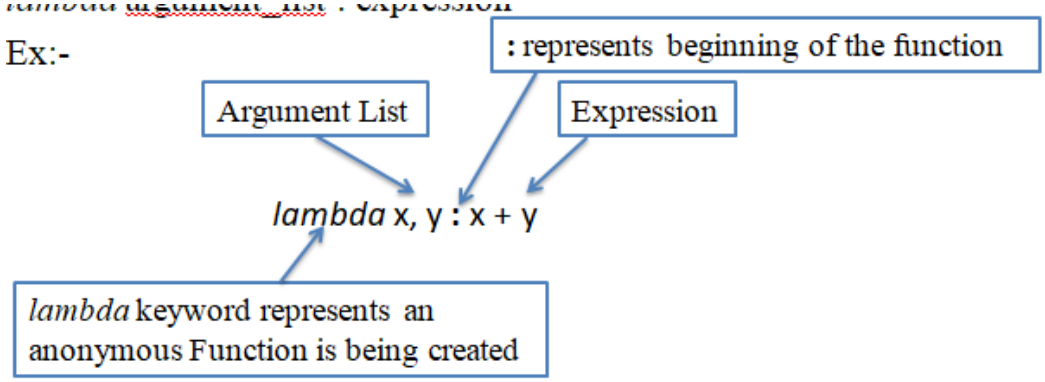
Syntax:-
***lambda* argument_list : expression**

Ex:-
```
lambda x : print(x)
lambda x, y : x + y
```

Creating a Lambda Function

Syntax:-
lambda argument_list : expression

Ex:-




```
#Example 1 Single Argument
show = lambda x : print(x)
show(5)
```

```
#Example 2 Two Arguments
add = lambda x,y : (x+y)
print(add(5, 2))
```

```
#Example 3 Return Multiple
add_sub = lambda x,y : (x+y, x-y)
a, s = add_sub(5, 2)
print(a)
print(s)



#Example 2 with Default Argument
add = lambda x,y=3 : (x+y)
print(add(5))
```

List

- A list represents a group of elements.
- Lists are very similar to array but there is major difference, an array can store only one type of elements whereas a list can store different type of elements.
- Lists are mutable so we can modify it's element.
- A list can store different types of elements which can be modified.
- Lists are dynamic which means size is not fixed.
- Lists are represented using square bracket [ ].

Ex:- a = [10, 20, -50, 21.3, 'IndiaShow']
Ex:- a = [10, 20, 50]

Creating a List

A list is similar to an array that consists of a group of elements or items.
Syntax:- list_name = [element1, element2, …..]

Ex:- a = [10, 20, -50, 21.3, 'IndiaShow']

Creating an Empty List

Syntax:- list_name = [ ]

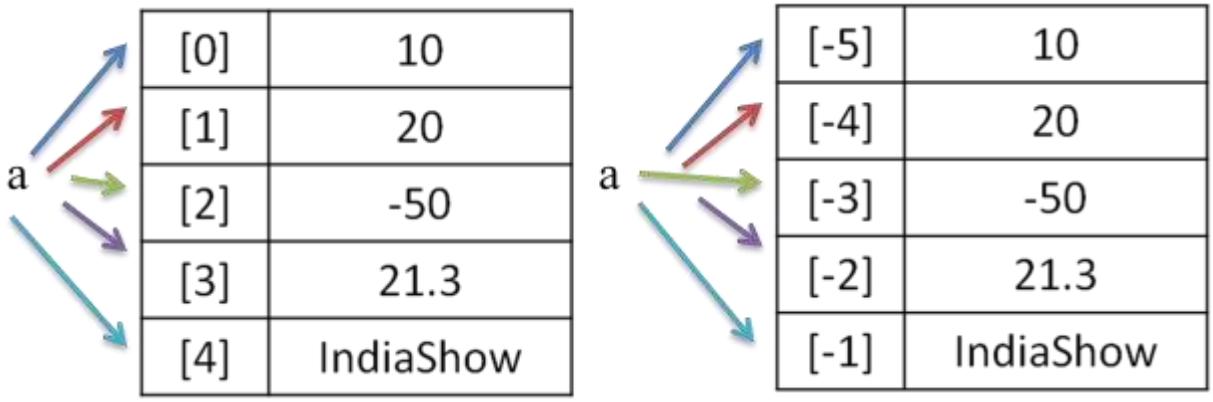
Ex:- a = [ ]

Index

An index represents the position number of an list's element.

The index start from 0 on wards and written inside square braces.

Ex:- a = [10, 20, -50, 21.3, 'IndiaShow']



Accessing List's Element

```
a = [10, 20, -50, 21.3, 'IndiaShow']
print(a[0])
print(a[1])
print(a[2])
print(a[3])
print(a[4])
```

Modifying or Updating Element

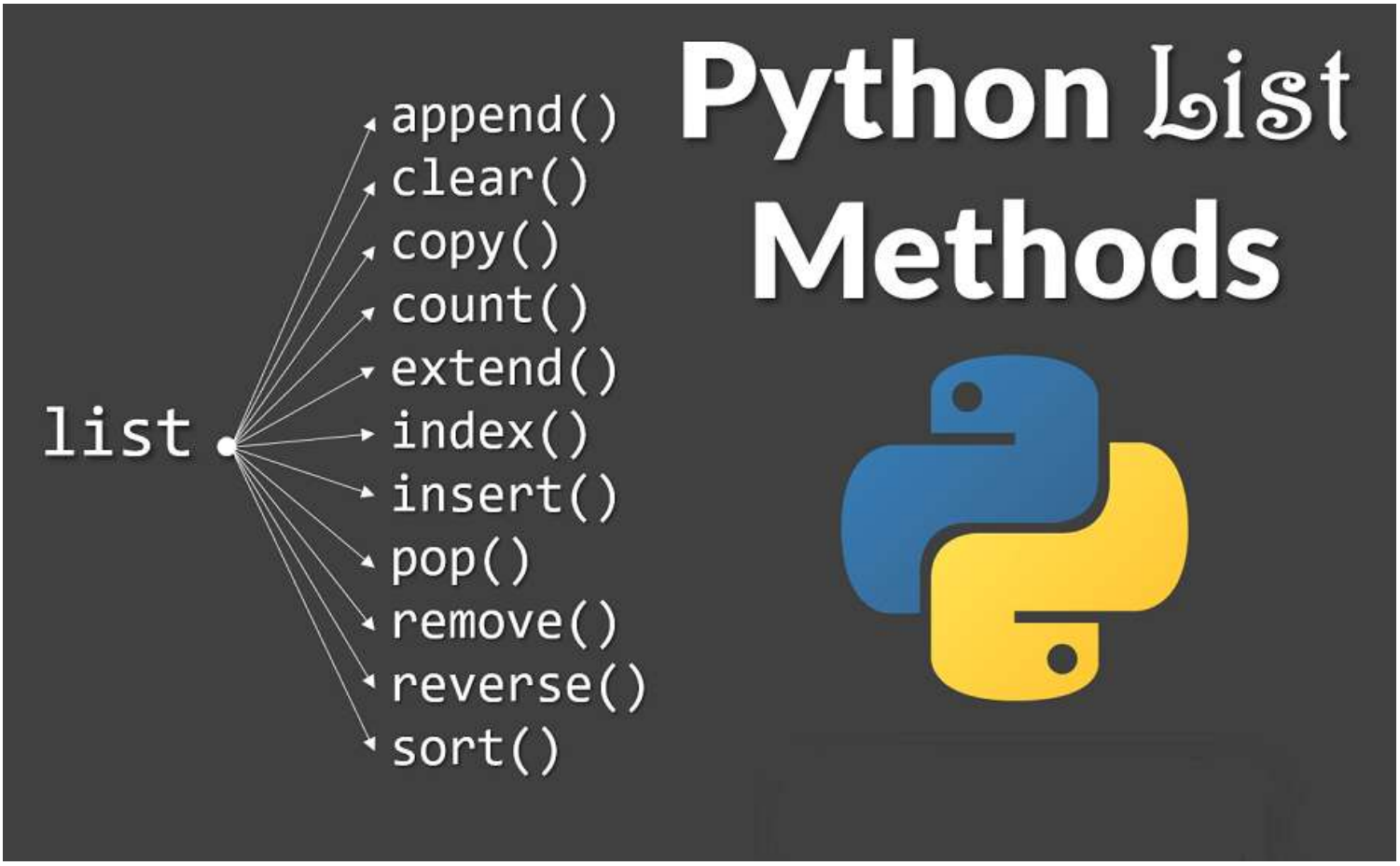
Lists are mutable so we can modify it's element.

```
a = [10, 20, -50, 21.3, 'IndiaShow']
a[1] = 40
```

```
a = [10, 20, -50, 21.3, ' IndiaShow ']
print(a,id(a))          #[10,20,-50,21.3,'IndiaShow'] 1714929290688
print(a[1])             #20
a[1] = 40
print(a[1])             #40
print(a, id(a))         #[10,40,-50,21.3,'IndiaShow'] 1714929290688
```

Accessing using loops

```
# Accessing List using for loop
a = [10, 20, -50, 21.3, ' IndiaShow']


#without index
print("Accessing List using for Loop without index")
for element in a:
     print(element)
print()


#With Index
print("Accessing List using for Loop with index")
n = len(a)
for i in range(n):
     print(i, "=", a[i])



print("Accessing List using while Loop")
n = len(a)
i = 0
while i < n:
     print(i, "=", a[i])
     i+=1



# Deleting single element of List
print("After Deletion:")
del a[2]        #50 gets deleted
print(a)
print()



# Deleting entire List
del a
print(a)  # It will show error as List a has been deleted
```




```
# Append Method
a = [10, 20, -50, 21.3, 'IndiaShows']


print("Before Appending:")
for element in a:
     print (element)


# Appending an element
a.append(100)
print()
print("After Appending")
for element in a:
     print (element)
```

	Description	Code example	Diagram
append()	Adds an element to the end of the list	letters = ['a', 'b', 'c'] letters.append('d')	
extend()	Adds the elements of a list to the end of another list	letters = ['a', 'b', 'c'] letters.extend(['d', 'e', 'f'])	
insert()	Adds an element at a specific index	letters = ['a', 'b', 'c'] letters.insert(1, 'x')	
remove()	Removes the first occurrence of an element	letters = ['a', 'b', 'c', 'b'] letters.remove('b')	
pop()	Removes and returns the element at a given index	letters = ['a', 'b', 'c', 'd'] letter = letters.pop(1)	
count()	Returns the number of times a value appears in a list	letters = ['c', 'b', 'c', 'c', 'd'] print(letters.count('c'))	
sort() @NikkiSiapno	Sorts the list in ascending order	letters = ['c', 'a', 'd', 'a', 'b'] letters.sort()	
reverse() @ChrisStaud	Reverses the order of elements in the list	letters = ['a', 'b', 'c'] letters.reverse()	

Tuple

- Tuple – A tuple contains a group of elements which can be same or different types.
- Tuples are immutable.
- It is similar to List but Tuples are read-only which means we can not modify it's element.
- Tuples are used to store data which should not be modified.
- It occupies less memory compare to list.
- Tuples are represented using parentheses ( ).
- Ex:- a = (10, 20, -50, 21.3, 'IndiaShow')

Creating Tuple

We can create tuple by writing elements separated by commas inside parentheses.

With one Element

b = (10) ← It will become integer

With Multiple Elements

d = (10, 20, 30, 40)

e = (10, 20, -50, 21.3, 'IndiaShows')

f = (10, 20, -50, 21.3, 'IndiaShows') ← It will become a tuple

Python Tuples Packing

You can also create a Python tuple without parentheses.
This is called tuple packing.

```
b= 1, 2.0, 'three'
print(b)
```

Python Tuples Unpacking

Python tuple unpacking is when you assign values from a tuple to a sequence of variables in python.

```
>>> percentages=(99,95,90,89,93,96)
>>> a,b,c,d,e,f=percentages
>>> c
```

Output

90

Accessing Tuple's Element

Accessing using for & while loop

Modifying Element - Tuples are immutable so it is not possible to modify, update or delete it's element
Deleting Tuple - You can delete entire tuple but not an element of tuple.

Slicing on Tuple

Slicing on tuple can be used to retrieve a piece of the tuple that contains a group of elements. Slicing is useful to retrieve a range of elements.
Syntax:-
new_tuple_name = tuple_name[start:stop:stepsize]

```
a = [10, 20, -50, 21.3, 'IndiaShows']
b = a[0:5:1]
print(b)
```

Set Type

A set is an unordered collection of elements much like a set in mathematics.
The order of elements is not maintained in the sets. It means the elements may not appear in the same order as they are entered into the set.
A set does not accept duplicate elements.
Set is mutable so we can modify it.
Sets are unordered so we can not access its element using index.
Sets are represented using curly brackets { }
a = {10, 20, 30, "Students", "Raj", 40}

Creating a Set

A set is created by placing all the items (elements) inside curly braces {}, separated by comma. A set does not accept duplicate elements.
Elements can be of different types except mutable element, like list, set or dictionary.
Ex:-
a = {10, 20, 30}
a = {10, 20, 30, "Students", "Raj", 40}
a = {10, 20, 30, "Students", "Raj", 40, 10, 20}

Modifying Elements

Sets are mutable but as we can not access elements using index so we can not modify it

Python Set Methods

Input	Method	Output
{5, 'Hello', 3.2}	.add(2)	{2, 'Hello', 3.2, 5}
{5, 'Hello', 3.2}	.discard(3.2)	{'Hello', 5}
{5, 'Hello', 3.2}	.remove(3.2)	{'Hello', 5}
{5, 'Hello', 3.2}	.pop()	{3.2, 5}
{5, 2, 8}	.intersection({5, 1})	{5}
{5, 2, 8}	.difference({5, 1})	{8, 2}
{1, -2}	.issubset({-2, 1, 3})	True
{-2, 1, 3}	.issuperset({1, -2})	True
{-2, 1, 3}	.isdisjoint({4, 6})	True
{1, 2, 3}	.union({3, 1})	{1, 2, 3}
{1, 2, 3}	.update({'a', 'b'})	{1, 2, 3, 'b', 'a'}

```
#Creating Empty Set
x = set()

#Cretaing Set with elements
a = {10, 20, 'Students', 'Raj', 40}

# Cant access using index
# print(a[0])    # It will show error
print(a)
print(id(a))
print()

# Duplicates are avoided
b = {10, 20, 'Students', 'Raj', 40, 10, 10}
print(b)
print()
```

```
# Adding one Element
print("Adding one Element:")
a.add('Python')
print(a)
print(id(a))
print()

#Adding Multiple Elements
print("Adding Multiple Element:")
a.update([101, 102, 103])
print(a)
print(id(a))
print()
```

```
#Deleting Element using discard() method
print("Removing Element using discard:")
a.discard('Students')
print(a)
print(id(a))
print()

#Deleting Element using remove() method
print("Removing Element using remove:")
a.remove('Raj')
print(a)
print(id(a))
print()
```

```
# Set Methods

a = {'Rahul', 'Raj', 'Sonam', 'Rani'}
b = {'Sumit', 'Rahul', 'Rani', 'Python', 'Java'}

print("A:",a)
print("B:",b)
print()

# intersection() Method
        # returns item which exists in both sets
ism = a.intersection(b)
print("Intersection:", ism)
print()

# union() Method
        # Returns all item from original set and all items
from specified set
um = a.union(b)
print("Union:", um)
print()
```

```
# difference() Method
        # Returns items that exist only in first set,
and not in both sets
dm = a.difference(b)
print("Difference:", dm)
print()

# issubset() Method
        #Returns True if all items in the set exists
in the specified set
isub = a.issubset(b)
print("issubset:", isub)
print()

# issuperset() Method
# Returns True if all items in the specified set exists in
the original set
isup = a.issuperset(b)
print("issuperset:", isup)
print()
```

Dictionary

A Dictionary represents a group of elements in the form of key value pairs.
Dictionary in Python is an unordered collection.
Dictionaries are mutable so we can modify it's item, without changing their identity.
Dictionaries are represented using curly bracket { }.
Ex:-
stu = {101: 'Rahul', 102: 'Raj', 103: 'Sonam' }
fees= {'rahul':2000, 'raj':3000, 'sonam':8000}

Creating a Dictionary

A dictionary is created in the form of key-value pair where keys can't be repeated and must be immutable and values can be of any datatype and can be duplicated.
keys are case sensitive.
Syntax:- dict_name = {key1:value1, key2:value2,…}

Key Rules

While writing key we must follow the following rules:

- Key should be unique.
- If we mention same key again, the old key will be overwritten.
- Key should be immutable type ex:- integer, string or tuple.
- We can not use list or dictionary as key.

Accessing Dictionary

We can access the value of a dictionary by referring to its key name, inside square brackets.

```
stu = {101: 'Rahul', 102: 'Raj', 103: 'Sonam' }
fees = {'rahul':2000, 'raj':3000, 'sonam':8000}
print(stu[101])
print(stu[102])
print(stu[103])

print(fees['rahul'])
print(fees['raj'])
print(fees['sonam'])
```

Modifying

We can modify the existing value of key by assigning a new value.

```
stu = {101: 'Rahul', 102: 'Raj', 103: 'Sonam' }
stu[102] = 'Python'
print(stu[102])
```

Method	Example	Description

a.copy( )	a={1: 'ONE', 2: 'two', 3: 'three'}
	>>> b=a.copy()
	>>> print(b)
	{1: 'ONE', 2: 'two', 3: 'three'}
		It returns copy of the dictionary. here copy of dictionary 'a' get stored in to dictionary 'b'

a.items()	>>> a.items() dict_items([(1, 'ONE'), (2, 'two'), (3, 'three')])	Return a new view of the dictionary's items. It displays a list of dictionary's (key, value) tuple pairs.
a.keys()	>>> a.keys() dict_keys([1, 2, 3])	It displays list of keys in a dictionary
a.values()	>>> a.values() dict_values(['ONE', 'two', 'three'])	It displays list of values in dictionary

a.pop(key)	>>> a.pop(3)
	'three'
	>>> print(a)
	{1: 'ONE', 2: 'two'}
		Remove the element with *key* and return its value from the dictionary.

setdefault(key,value)	>>> a.setdefault(3,"three")
	'three'
	>>> print(a)
	{1: 'ONE', 2: 'two', 3: 'three'}
	>>> a.setdefault(2)
	'two'
		If key is in the dictionary, return its value. If key is not present, insert key with a value of dictionary and return dictionary.

a.update(dictionary)	>>> b={4:"four"}
	>>> a.update(b)
	>>> print(a)
	{1: 'ONE', 2: 'two', 3: 'three', 4: 'four'}
		It will add the dictionary with the existing dictionary

fromkeys()	>>> key={"apple","ball"}
	>>> value="for kids"
	>>> d=dict.fromkeys(key,value)
	>>> print(d)
	{'apple': 'for kids', 'ball': 'for kids'}
		It creates a dictionary from key and values.

len(a)	a={1: 'ONE', 2: 'two', 3: 'three'} >>>lena(a) 3	It returns the length of the list.
clear()	a={1: 'ONE', 2: 'two', 3: 'three'} >>>a.clear() >>>print(a) >>>{ }	Remove all elements form the dictionary.
del(a)	a={1: 'ONE', 2: 'two', 3: 'three'} >>> del(a)	It will delete the entire dictionary.

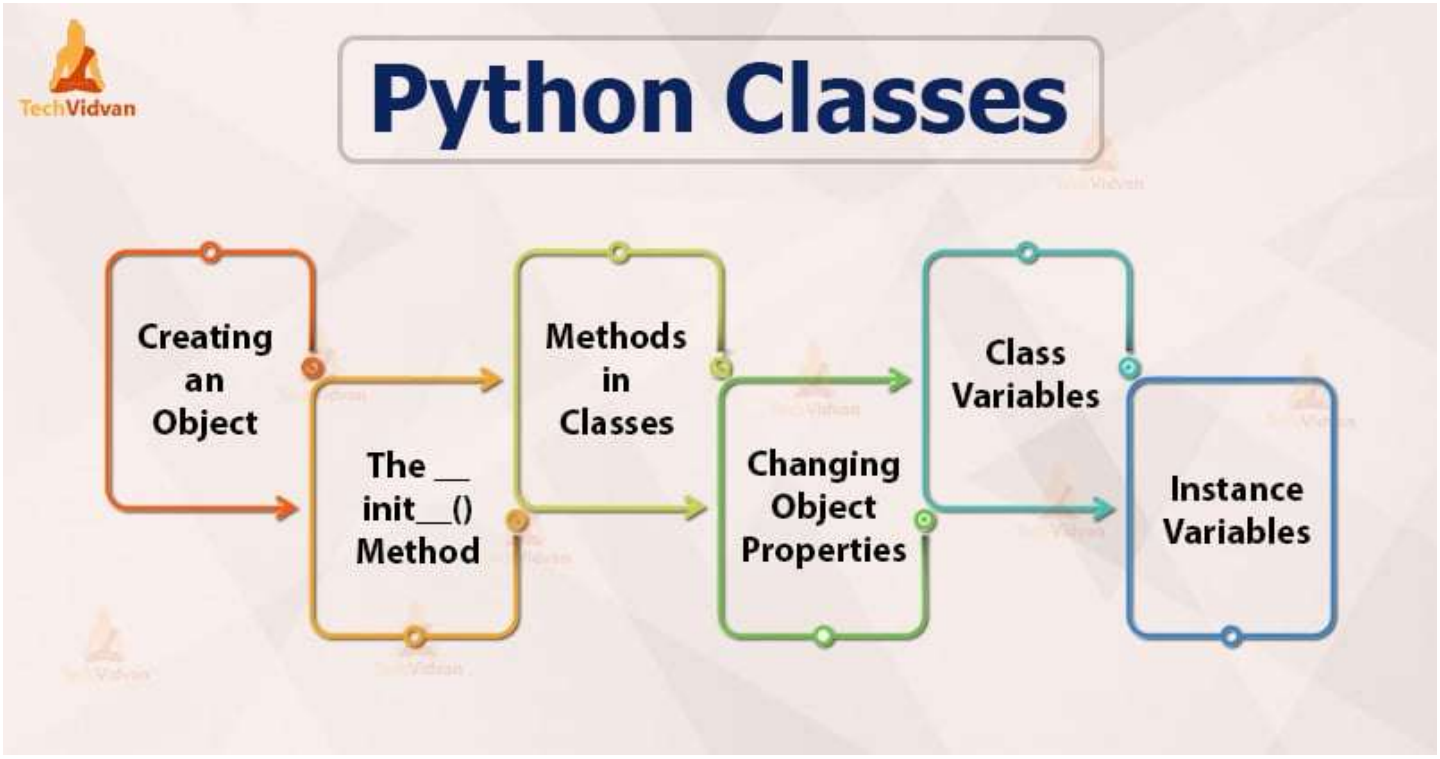
LISTS VS. TUPLES VS. SETS VS. DICTIONARIES

	Lists	Tuples	Sets	Dictionaries
Ordering	Ordered	Ordered	Unordered	Ordered Unordered before Python 3.7
Indexing	Indexed	Indexed	Not Indexed	Keyed
Mutability	Mutable	Immutable	Mutable Only Adding and Removing	Mutable
Duplicates Allowed	Yes	Yes	No	Yes Only in values and not in keys
Types Allowed	Mutable and Immutable	Mutable and Immutable	Only Immutable	Only Immutable In keys

Object Oriented Programming

Object-oriented programming (OOP) may be a **programming model** that organizes **software design** around **data**, or **objects**, instead of **functions** and **logic**.

An object are often defined as a **knowledge field** that has **unique attributes** and **behavior**



Class

Python is an **object-oriented language** and everything in it is an **object**.

By using Python **classes** and **objects**, we can **model** the **real world**.

A **class** is like a **blueprint** for **objects** – it has **no values** itself.

We can have **multiple objects** of **one class**.

A Python class is a group of attributes and methods.

What is Attribute ?
Attributes are represented by variable that contains data.

What is Method?
Method performs an action or task. It is similar to function.

ClassName Rules :

- The class name can be any valid identifier.
- It can't be Python reserved word.
- A valid class name starts with a letter, followed by any number of letter, numbers or underscores.
- A class name generally starts with Capital Letter.

How to Create Class

To define a **class**, we use the **class keyword.**
This was an **empty class.**

```
class Mobile:
```

Object

- Object is class type variable or class instance.
- To use a class, we should create an object to the class.
- Instance/Local variable creation represents allotting memory necessary to store the actual data of the variables.
- Each time you create an object of a class, a copy of each variables defined in the class is created.
- In other words you can say that each object of a class has its own copy of data members defined in the class.
- There is **no** need to **use** the **new keyword** for **creating** objects in **Python**. Just use this **class constructor**:

Syntax: -

```
object_name = class_name()
object_name = class_name(arg)
```

```
>>> Mobile = Apple()
```

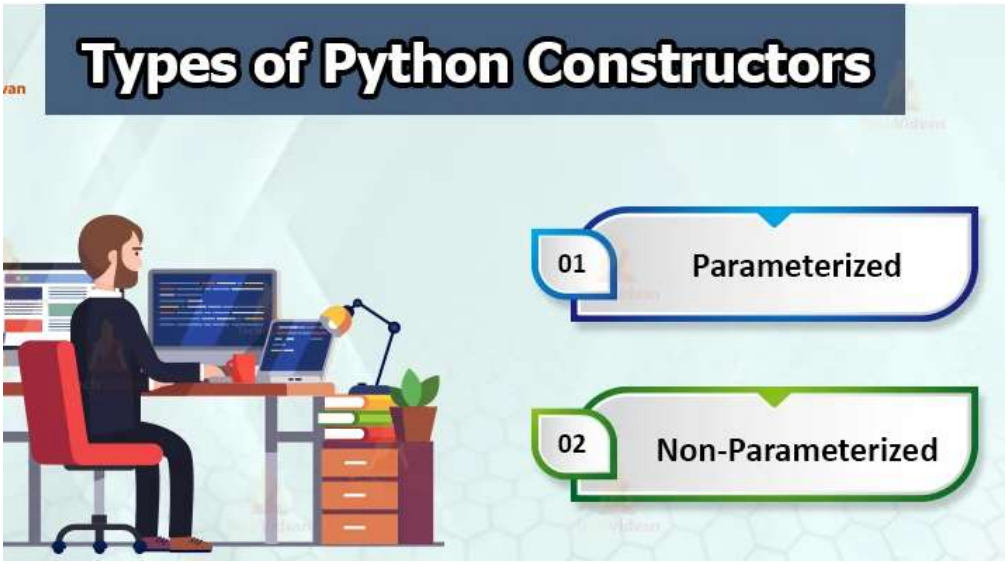
Constructor

Python supports a special type of method called constructor for initializing the instance variable of a class.

A class constructor, if defined is called whenever a program creates an object of that class.

A constructor is called only once at the time of creating an instance.

If two instances are created for a class, the constructor will be called once for each instance.



Constructors in Python can be of **two types:**

a. Parameterized Constructor

Parameterized constructors are ones which have **parameters(other than self)** defined in the **__init__** method's **parameter list.**

This type of constructor can take **arguments** from the user.

b. Non-parameterized Constructor

- It is also known as the **default constructor.**
- The **__init__** method includes a **single parameter** self.
- No other parameters are present in **__init__'s parameter list.**
- Consequently, this constructor takes **no arguments** while **creating** a **new object.**
- **Non-parameterized constructors** assign default values to the **attributes** of the **class.**

The __init__() Method

This is a **magic method (dunder method)** which we can use to **initialize variables** for **classes (objects).**

Every **class** has **__init__** and this is **executed** when we **instantiate** the **class.**

You can also use this method to do **anything** you want to do when the **object** is being **created.**

We do not call this method explicitly.

self -

self is a default variable that contains the memory address of the current object.

This variable is used to refer all the instance variable and method.

When we create object of a class, the object name contains the memory location of the object.

This memory location is internally passed to *self*, as *self* knows the memory address of the object so we can access variable and method of object.

self is the first argument to any object method because the first argument is always the object reference.

This is automatic, whether you call it *self* or not.

We can even use any other variable name instead of self variable

Ex--

```
def __init__(self):      #constructor
def show_model(self):    #method
```

Constructor without Parameter

```
# Constructor
class Mobile:
     def __init__(self):
          print("Mobile Constructor Called")

realme = Mobile()
```

```
class Mobile:
     # Constructor without parameter
     def __init__(self):
          self.model = 'RealMe X'

     def show_model(self):
          print("Model:", self.model)


realme = Mobile()
realme.show_model()
```

Constructor with Parameter

```
# Constructor
class Mobile:
     # Constructor
     def __init__(self, m, v=80):
          self.model = m
          self.volumn = v

     def show_model(self, p):
          price = p                # Local Varaible
          print("Model:", self.model, "and Price:", price)
          print("Volumn:", self.volumn)

# Passing Argument to Constructor
realme = Mobile('Realme X')

# Accessing Method from outside Class
realme.show_model(1000)
print()

redmi = Mobile('Redmi 7s', 50)
redmi.show_model(2000)
```

Type of Methods

- Instance Methods
  - Accessor Methods
  - Mutator Methods
- Class Methods
- Static Methods

Instance Method

Instance methods are the methods which act upon the instance variables of the class.

Instance method need to know the memory address of the instance which is provided through *self* variable by default as first parameter for the instance method.

Instance Method without Parameter

```
# Instance Variable with Instance Method
class Mobile:
     def __init__(self):
          self.model = 'RealMe X'          # Instance Variable

     def show_model(self):                 # Instance Method
          print(self.model)                # Accessing Instance Variable


realme = Mobile()
realme.show_model()

# Accessing Instance Variable from Outside Class
r = realme.model
print("Outside Class:", r)
```

Instance Method with Parameter

```
# Instance Method with Parameter
class Mobile:
     def __init__(self):
          self.model = 'RealMe X'                    # Instance Variable

     def show_model(self, p):                  # Instance Method with
Parameter
          self.price = p
          # Accessing Instance Variable
          print("Model:", self.model, "Price:", self.price)

realme = Mobile()
realme.show_model(1000)                                  # Calling
Instance Method
```

Accessor Method

This method is used to access or read data of the variables.
This method do not modify the data in the variable.
This is also called as getter method.

Ex:-

```
def get_value(self):
def get_result(self):
def get_name(self):
def get_id(self):
```

```
# Instance Method - Accessor Method / Getter Method
class Mobile:
     def __init__(self):
          self.model = 'RealMe X'                    # Instance Variable

     def get_model(self):                            # Accessor Method

          return self.model

realme = Mobile()
m = realme.get_model()                          # Calling Accessor Method
print(m)
```

Mutator Method

This method is used to access or read and modify data of the variables.
This method modify the data in the variable.

This is also called as setter method.

Ex:-

```
def set_value(self):
def set_result(self):
def set_name(self):
def set_id(self):
```

```
# Instance Method - Mutator Method / Setter Method
class Mobile:
     def __init__(self):
          self.model = 'RealMe X'                    # Instance Variable

     def set_model(self):                            # Mutator Method

          self.model = 'RealMe 2'

realme = Mobile()

# Before setting
print(realme.model)

# After Setting
realme.set_model()                              # Calling Mutator Method
print(realme.model)
```

```
# Instance Method - Mutator Method / Setter Method
class Mobile:
     def set_model(self, m):                         # Mutator Method
              self.model = m

realme = Mobile()

realme.set_model('RealMe X')                    # Calling Mutator Method
print(realme.model)
```

Class Methods

Class methods are the methods which act upon the class variables or static variable of the class.

Decorator @classmethod need to write above the class method.

By default, the first parameter of class method is cls which refers to the class itself.

Class Method without Parameter

```
# Class Method w/o Parameter
class Mobile:
     @classmethod                          # Decorator
     def show_model(cls):                  # Class Method
          print("RealMe X")

realme = Mobile()
Mobile.show_model()                        # Calling Class Method
```

```
# Class Method w/o Parameter
class Mobile:
     fp = 'Yes'                                  # Class Variable

     @classmethod
     def show_model(cls):                  # Class Method
          # Accessing Class Variable
          print("Fingerprint Scaner:", cls.fp)

realme = Mobile()
Mobile.show_model()                        # Calling Class Method
```

Class Method with Parameter

```
# Class Method with Parameter
class Mobile:
     fp = 'Yes'                          # Class Variable

     @classmethod                  # Decorator
     def show_model(cls, r):             # Class Method
          cls.ram = r
          # Accessing Class Variable
          print("Fingerprint Scaner:", cls.fp, "RAM:", cls.ram)

realme = Mobile()
Mobile.show_model('4GB')            # Calling Class Method
```

Static Methods

Static Methods are used when some processing is related to the class but does not need the class or its instances to perform any work.

We use static method when we want to pass some values from outside and perform some action in the method.

Decorator @staticmethod need to write above the static method.

Static Method without Parameter

```
# Static Method w/o Parameter
class Mobile:
     @staticmethod                  # Decorator
     def show_model():              # Static Method
          print("RealMe X")

realme = Mobile()
Mobile.show_model()                 # Calling Static Method
```

```
# Static Method w/o Parameter
class Mobile:
     fp = 'Yes'                                 # Class Variable

     @staticmethod
     def show_model():                      # Static Method
          # Accessing Class Variable
          print("Fingerprint Scaner:", Mobile.fp)

realme = Mobile()
Mobile.show_model()                         # Calling Static Method
```

Static Method with Parameter

```
# Static Method with Parameter
class Mobile:

     @staticmethod
     def show_model(m, p):                    # Static Method
          model = m
          price = p
          print("Model:", model, "Price", price)

realme = Mobile()
Mobile.show_model('RealMe X', 1000)          # Calling Static Method
```

Type of Variable

- Instance Variable/Local Variable
- Class Variable / Static Variable

Instance Variable

Instance variables are the variables whose separate copy is created in every object.

Instance variables are defined and initialized using a constructor with self parameter.

To access instance variable, we need instance methods with self as first parameter then we can access instance variable using self.variable_name

```
class Mobile:
    def __init__(self):
        self.model = 'RealMe X'
    def show_model(self):
        self.model
realme = Mobile( )
```

Instance Variable

Instance Method

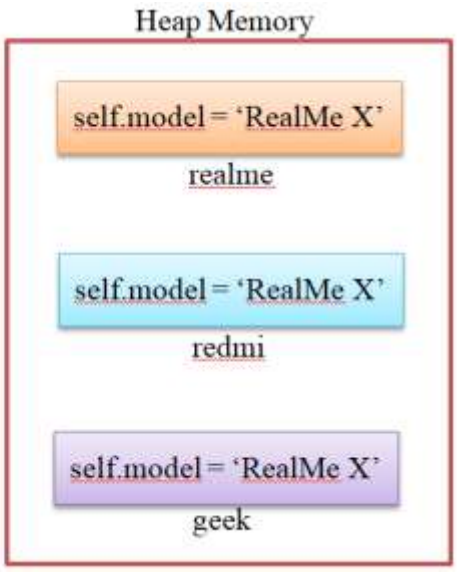
Accessing Instance Variable

Instance variables are the variables whose separate copy is created in every object.

If we modify the copy of Instance variable in an instance, it will not effect all the copies in the other instance.

```
class Mobile:
    def __init__(self):
        self.model = 'RealMe X'

    def show_model(self):
        print(self.model)

realme = Mobile( )
redmi = Mobile( )
geek = Mobile( )
```

Instance Variable



Class Variable / Static Variable

Class variables are the variables whose single copy is available to all the instance of the class.

If we modify the copy of class variable in an instance, it will effect all the copies in the other instance.

Ex:-

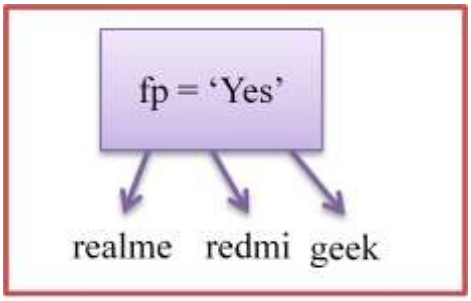
```
class Mobile:
     name = 'Bahubali'    #class variable/Global Variable

     def __init__(self):
          self.model = 'RealMe X'

    def show_model(self):
         print(self.model)

realme = Mobile( )
```

With Class Method

To access class variable, we need class methods with cls as first parameter then we can access class variable using cls.variable_name

Class variables are the variables whose single copy is available to all the instance of the class. If we modify the copy of class variable in an instance, it will effect all the copies in the other instance.

```
class Mobile:
     name = 'Hi hello'   #class variable/Global Variable

     def __init__(self):
          self.model = 'RealMe X'

     @classmethod
     def is_name(cls):
          print(cls.name) #Accessing Class Variable inside Class Method

realme = Mobile( )
print(Mobile.name)  #access var using classname
Mobile.is_name()    #access using classmethod
realme.is_name()    #access using objname

readme = Mobile( )
readme.is_name()    #access using objname

#geek = Mobile( )
```



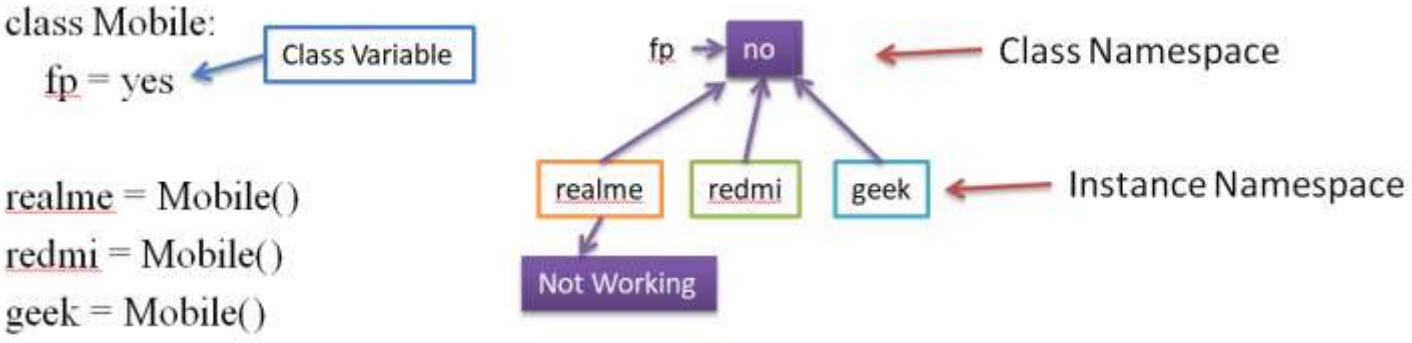
Heap Memory

Namespace

In Python, Namespace represents a memory block where names are mapped to objects.

Class Namespace –
A class maintains it's own namespace known as class namespace.
In the class namespace, the names are mapped to class variables.

Instance Namespace –
Every instance have it's own namespace known as instance namespace.
In the instance namespace, the names are mapped to instance variables.



Mobile.fp	# yes	Mobile.fp = no		realme.fp = Not Working	
realme.fp	# yes	Mobile.fp	# no	Mobile.fp	# no
redmi.fp	# yes	realme.fp	# no	realme.fp	# Not Working
geek.fp	# yes	redmi.fp	# no	redmi.fp	# no
		geek.fp	# no	geek.fp	# no

```python
class Mobile:
     name = 'Student'    #class variable/Global Variable


realme = Mobile()
redmi  = Mobile()
geek   = Mobile()

print(Mobile.name)  #Student
print(realme.name)  #Student
print(redmi.name)   #Student
print(geek.name)    #Student

Mobile.name="Bahubali"   #modifying the mobile class
print(Mobile.name)  #Bahubali
print(realme.name)  #Bahubali
print(redmi.name)   #Bahubali
print(geek.name)    #Bahubali

#-----------------------------------------------
realme.name='work'    #Modifying the Realme Obj
print(realme.name)    #work
print(redmi.name)     #Bahubali
print(geek.name)      #Bahubali

#-----------------------------------------------
redmi.name='Reading'    #Modifying the Realme Obj
print(realme.name)    #work
print(redmi.name)     #Reading
print(geek.name)      #Bahubali
```

Inheritance

The mechanism of deriving a new class from an old one (existing class) such that the new class inherit all the members (variables and methods) of old class is called inheritance or derivation.

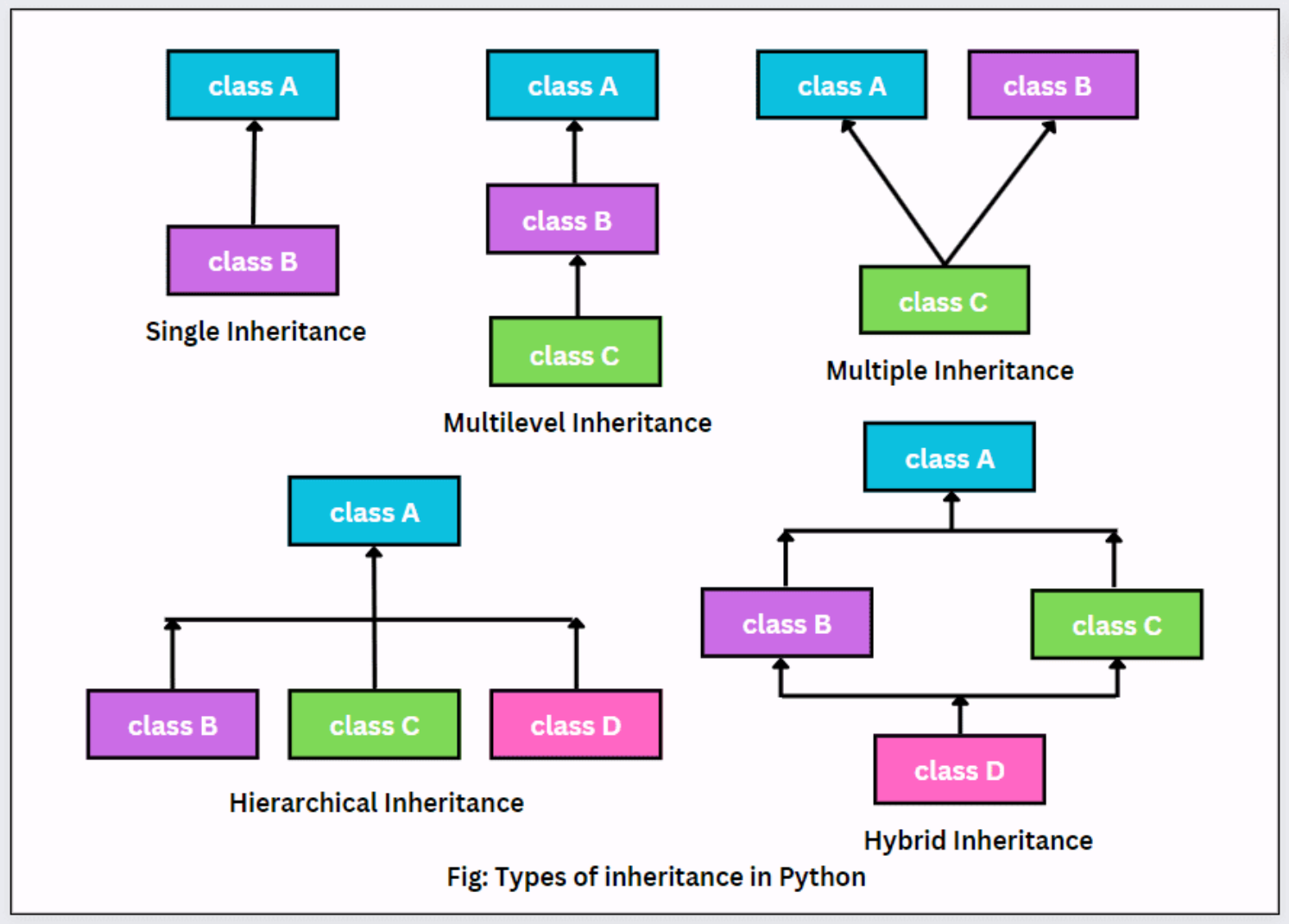


Fig: Types of inheritance in Python

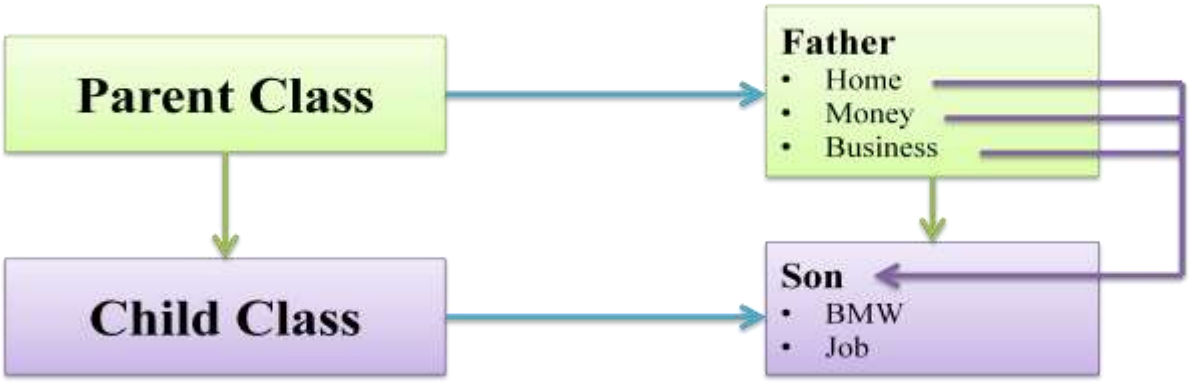
Super Class and Sub Class

The old class is referred to as the Super class and the new one is called the Sub class.

- Parent Class - Base Class or Super Class
- Child Class - Derived Class or Sub Class

- All classes in python are built from a single super class called 'object' so whenever we create a class in python, object will become super class for them internally.
  class Mobile(object):
  class Mobile:

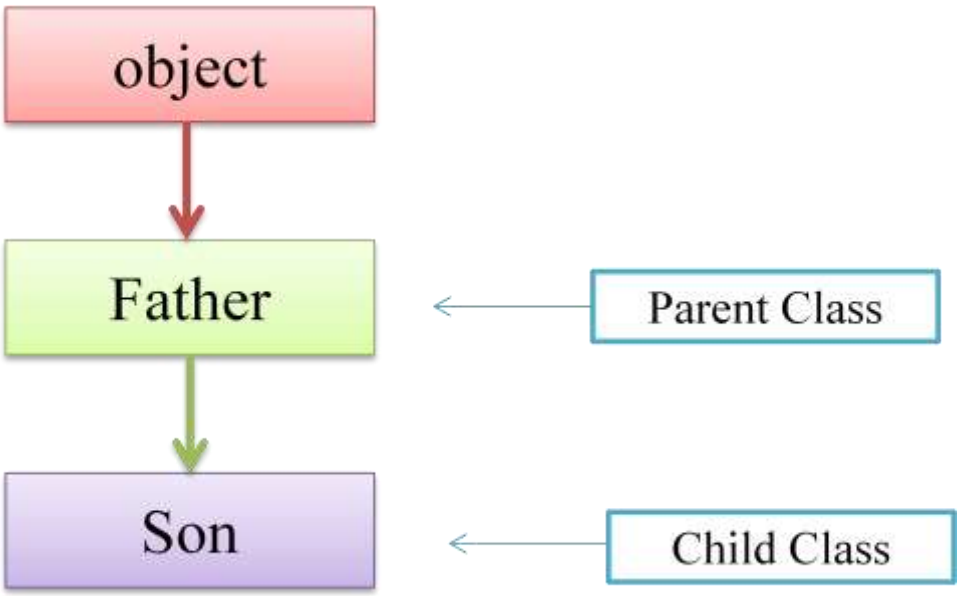
- The main advantage of inheritance is code reusability.



Single Inheritance

If a class is derived from one base class (Parent Class), it is called Single Inheritance.

- We can access Parent Class Variables and Methods using Child Class Object.
- We can also access Parent Class Variables and Methods using Parent Class Object.
- We can not access Child Class Variables and Methods using Parent Class Object.



Constructor in Inheritance

By default, The constructor in the parent class is available to the child class.

```
class Parent:
     def __init__(self):
          self.money = 2000
     print("Father Class Constructor")

#-----------------------------------
class Child(Parent):
     def disp(self):
          print("Son Class  Instance Method:", self.money)

c = Child()     #Father Class Constructor
c.disp()        #Son Class  Instance Method: 2000
```

Constructor Overriding

If we write constructor in the both classes, parent class and child class then the parent class constructor is not available to the child class.

In this case only child class constructor is accessible which means child class constructor is replacing parent class constructor.

Constructor overriding is used when programmer want to modify the existing behavior of a constructor.

```
class Father:
     def __init__(self):
          self.money = 2000
          print("Father Class Constructor")

class Son(Father):
     def __init__(self):
          self.money = 5000
          print("Son Class Constructor")

     def disp(self):
          print(self.money)

s = Son() #Son Class Constructor
s.disp()  #5000
```

Constructor with super( ) Method

If we write constructor in the both classes, parent class and child class then the parent class constructor is not available to the child class.

In this case only child class constructor is accessible which means child class constructor is replacing parent class constructor.

super() method is used to call parent class constructor or methods from the child class.

```
# Constructor with Super Method
class Father:                          # Parent Class
     def __init__(self):
          print("Father Class Constructor")

     def show(self):
          print("Father Class Instance Method")


class Son(Father):                  # Child Class
     def __init__(self):
          super().__init__()    # Calling Parent Class Constructor
          print("Son Class Constructor")


     def disp(self):
          print("Son Class Instance Method")

s = Son()
s.disp()
s.show()
```

output:

```
Father Class Constructor
Son Class Constructor
Son Class Instance Method
Father Class Instance Method
```

super() function

When dealing with **inheritance**, **super()** is a very **handy function**. **super()** is a **proxy object** which is used to **refer** to the **parent object**.

We can **call super()** method to **access** the **properties** or **methods** of the **parent class**.

```
class A:
    x=100
    def methodA(self):
        print("A class")

class B(A):
    def methodB(self):
        super().methodA() #A class
        print("B class")  #B class
        print(super().x)  #100

b = B()
b.methodB()
```

output:

```
A class
B class
100
```

Python Overriding Methods

Method overriding is an important concept in **object-oriented programming**. Method overriding allows us to **redefine** a method by **overriding** it.

For method overriding, we must satisfy **two conditions:**

- There should be a **parent-child relationship** between the **classes.**
- **Inheritance** is a must.
- The name of the **method** and the **parameters** should be the **same** in the **base** and **derived class** in order to **override** it.

What will happen when the method in the **base** and **derived class** are the same.

```
class A:
    def method(self):
        print("A class")

class B(A):
    def method(self):
        print("B class")

b = B()
b.method()      #B class
```

Overloading Methods in Python

If you have some experience with **object-oriented programming** or **other languages** like **C/C++** or **Java** then you might have used **method overloading.**

That is why it is important to understand that **Python doesn't support method overloading.**

```
def getDetails():
    print("Name: Default")

def getDetails(name):
    print("Name:", name)

getDetails("Siri")
getDetails()
```

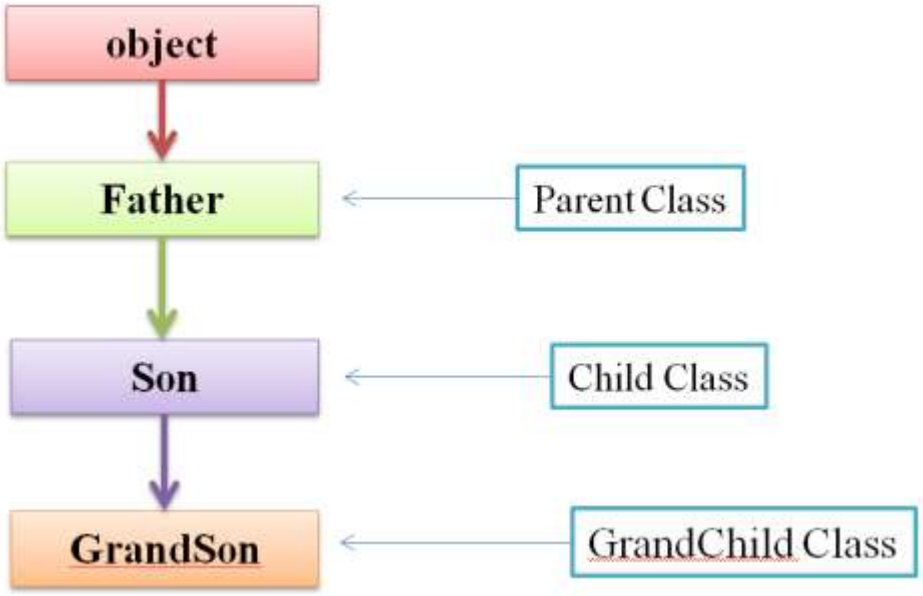
```
output:
Name: Siri

Traceback (most recent call last):
  File "c:\Users\user\Desktop\Simple\s.py", line 8, in <module>
    getDetails()
TypeError: getDetails() missing 1 required positional argument:
'name'
```

As you can see, the **getDetails("Siri")** function got **executed** but the **getDetails()** was **not executed.**

This is because **Python overwrites** the **function** with the **same name** and the **latter function** is **used.**

Multi-level Inheritance

In multi-level inheritance, the class inherits the feature of another derived class (Child Class).




```
class A:
    def methodA(self):
        print("A class")

class B(A):
    def methodB(self):
        print("B class")

class C(B):
    def methodC(self):
        print("C class")

c = C()
c.methodA()
c.methodB()
c.methodC()
```

ex-

```
# Multi-level Inheritance
class Father:
     def __init__(self):
          print("Father Class Constructor")
     def showF(self):
          print("Father Class Method")

class Son(Father):
     def __init__(self):
          print("Son Class Constructor")
     def showS(self):
          print("Son Class Method")

class GrandSon(Son):
     def __init__(self):
          print("GrandSon Class Constructor")
     def showG(self):
          print("GrandSon Class Method")

g = GrandSon()
g.showF()
g.showS()
g.showG()
```

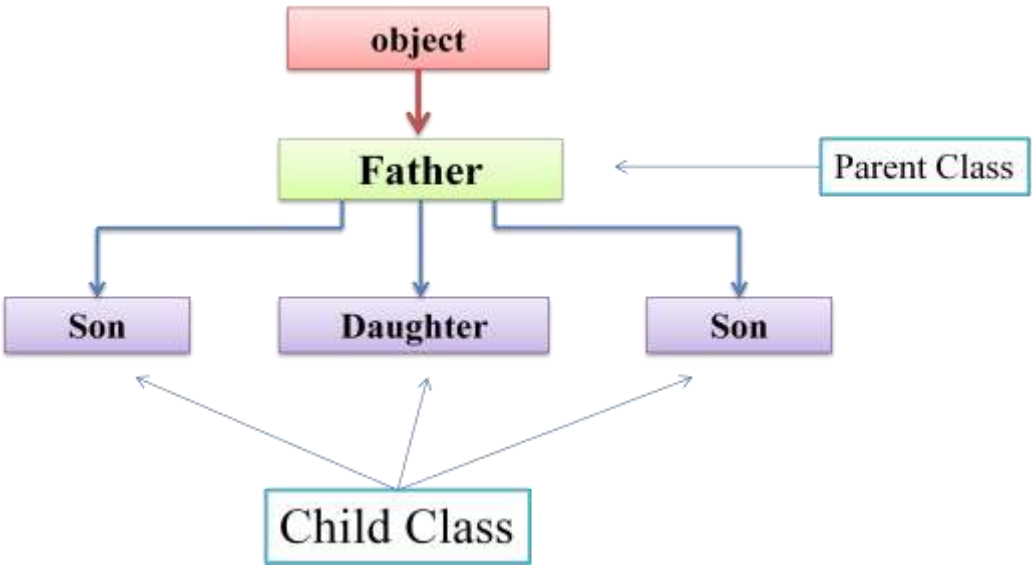
ex-

```
# Multi-level Inheritance
class Father:
     def __init__(self):
          print("Father Class Constructor")
     def showF(self):
          print("Father Class Method")

class Son(Father):
     def __init__(self):
          super().__init__()
          print("Son Class Constructor")
     def showS(self):
          print("Son Class Method")

class GrandSon(Son):
     def __init__(self):
          super().__init__()
          print("GrandSon Class Constructor")
     def showG(self):
          print("GrandSon Class Method")

g = GrandSon()
g.showF()
g.showS()
g.showG()
```

Output:

```
Father Class Constructor
Son Class Constructor
GrandSon Class Constructor
Father Class Method
Son Class Method
GrandSon Class Method
```

Hierarchical Inheritance

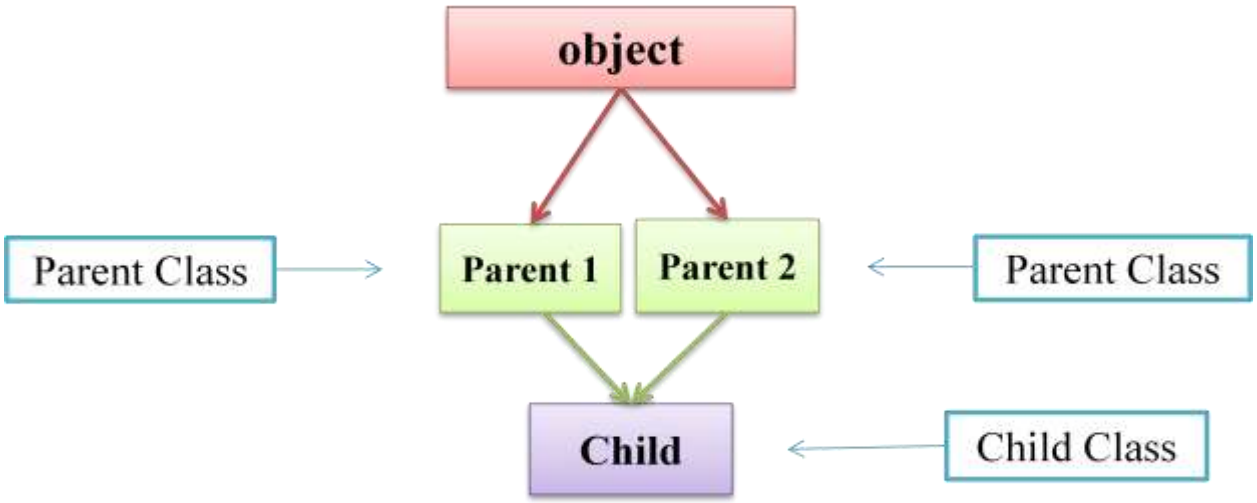



```
class Father:
     def __init__(self):
          print("Father Class Constructor")
     def showF(self):
          print("Father Class Method")

class Son(Father):
     def __init__(self):
          print("Son Class Constructor")
     def showS(self):
          print("Son Class Method")

class Daughter(Father):
     def __init__(self):
          print("Daughter Class Constructor")
     def showD(self):
          print("Daughter Class Method")
d = Daughter()
d.showF()
d.showD()
s = Son()
s.showF()
s.showS()

Output:
Daughter Class Constructor
Father Class Method
Daughter Class Method
Son Class Constructor
Father Class Method
Son Class Method
```

```
# Hierarchical Inheritance
class Father:
     def __init__(self):
          print("Father Class Constructor")
     def showF(self):
          print("Father Class Method")

class Son(Father):
     def __init__(self):
          super().__init__()
          print("Son Class Constructor")
     def showS(self):
          print("Son Class Method")

class Daughter(Father):
     def __init__(self):
          super().__init__()
          print("Daughter Class Constructor")
     def showD(self):
          print("Daughter Class Method")

d = Daughter()
d.showF()
d.showD()
s = Son()
s.showF()
s.showS()

Output:
Father Class Constructor
Daughter Class Constructor
Father Class Method
Daughter Class Method
Father Class Constructor
Son Class Constructor
Father Class Method
Son Class Method
```

Multiple Inheritance

If a class is derived from more than one parent class, then it is called multiple inheritance.




```
# Multiple Inheritance
class Father:
     def __init__(self):
          print("Father Class Constructor")
     def showF(self):
          print("Father Class Method")

class Mother:
     def __init__(self):
          print("Mother Class Constructor")
     def showM(self):
          print("Mother Class Method")

class Son(Father, Mother):
     def __init__(self):
          print("Son Class Constructor")
     def showS(self):
          print("Son Class Method")


s = Son()
s.showF()
s.showM()
s.showS()
```

ex-

```
# Multiple Inheritance
class Father:
     def __init__(self):
          print("Father Class Constructor")
     def showF(self):
          print("Father Class Method")

class Mother:
     def __init__(self):
          print("Mother Class Constructor")
     def showM(self):
          print("Mother Class Method")

class Son(Father, Mother):
     def __init__(self):
          super().__init__()                    # Calling Parent
Class Constructor
          print("Son Class Constructor")
     def showS(self):
          print("Son Class Method")


s = Son()
s.showF()
s.showM()
s.showS()
```

Output:

```
Father Class Constructor
Son Class Constructor
Father Class Method
Mother Class Method
Son Class Method
```

Ex-

```
# Multiple Inheritance
class Father:
     def __init__(self):
          super().__init__()                    # Calling Parent
Class Constructor
          print("Father Class Constructor")
     def showF(self):
          print("Father Class Method")

class Mother:
     def __init__(self):
          super().__init__()                    # Calling Parent
Class Constructor
          print("Mother Class Constructor")
     def showM(self):
          print("Mother Class Method")

class Son(Father, Mother):
     def __init__(self):
          super().__init__()                    # Calling Parent
Class Constructor
          print("Son Class Constructor")
     def showS(self):
          print("Son Class Method")
s = Son()
s.showF()
s.showM()
s.showS()
```

Output:

```
Mother Class Constructor
Father Class Constructor
Son Class Constructor
Father Class Method
Mother Class Method
Son Class Method
```

Polymorphism

Polymorphism is a word that came from two greek words,
poly means many and **morphos** means forms.

If a variable, object or method perform different behavior according to situation, it is called polymorphism.

- Duck Typing
- Operator Overloading
- Method Overloading
- Method Overriding

Duck Typing

In Python, we follow a principle - If 'it walks like a duck and talks like a duck, it must be a duck' which means python doesn't care about which class of object it is, if it is an object and required behavior is present for that object then it will work. The type of object is distinguished only at runtime. This is called as duck typing.

Python doesn't care about which class of object it is, in order to call an existing method on an object. If the method is defined on the object, then it will be called.

```
# Duck Typing
class Duck:
     def walk(self):
          print("thapak thapak thapak thapak")

class Horse:
     def walk(self):
          print("tabdak tabdak tabdak tabdak")

def myfunction(obj):
     obj.walk()

d = Duck()
myfunction(d)

h = Horse()
myfunction(h)
```

Strong Typing

We can check whether the object passed to the method has the method being invoked or not.
hasattr ( ) Function is used to check whether the object has a method or not.

Syntax:- hasattr(object, attribute)

Where attribute can be a method or variable.
If it is found in the object then this method returns True else False.

```
# Strong Typing
class Duck:
     def walk(self):
          print("thapak thapak thapak thapak")

class Horse:
     def walk(self):
          print("tabdak tabdak tabdak tabdak")

class Cat:
     def talk(self):
          print("Meow Meow")

def myfunction(obj):
     if hasattr(obj, 'walk'):
          obj.walk()
     if hasattr(obj, 'talk'):
          obj.talk()

d = Duck()
myfunction(d)

h = Horse()
myfunction(h)

c = Cat()
myfunction(c)
```

Method Overloading

When more than one method with the same name is defined in the same class, it is known as method overloading.
In python, If a method is written such that it can perform more than one task, it is called method overloading.

```
# This Method Overloading Concept is not available in Python
# So it will show error

class Myclass:
     def sum(self, a):
          print("1st Sum Method", a)

     def sum(self):
          print("2nd Sum Method")

obj = Myclass()
obj.sum()
obj.sum(10)
```

```
# Method Overloading
class Myclass:
     def sum(self, a=None, b=None, c=None):
          if a!=None and b!=None and c!=None:
               s = a + b + c
          elif a!=None and b!=None:
               s = a + b
          else:
               s = 'Provide at least Two Numbers'
          return s

obj = Myclass()
print(obj.sum(10, 20, 30))
```

Method Overriding

If we write method in the both classes, parent class and child class then the parent class's method is not available to the child class.

In this case only child class's method is accessible which means child class's method is replacing parent class's method.

Method overriding is used when programmer want to modify the existing behavior of a Method.

```
class Add:
     def result(self, a, b):
          print("Addition:", a+b)

class Multi(Add):
     def result(self, a, b):
          print("Multiplication:", a*b) #200

m = Multi()
m.result(10, 20)
```

Method with super( ) Method

If we write method in the both classes, parent class and child class then the parent class's method is not available to the child class.

In this case only child class's method is accessible which means child class's method is replacing parent class's method.

super ( ) method is used to call parent class's *constructor* or *methods* from the child class.

Syntax:- super().methodName()

```
# Method Overriding
class Add:
     def result(self, a, b):
          print('Addition:', a+b)

class Multi(Add):
     def result(self, a, b):
          super().result(10, 20)  #Calling Parent Class's Method
          print('Multiplication:', a*b)

m = Multi()
m.result(10, 20)
```

Module

A module is a file containing Python definitions and statements.
A module is a file containing group of variables, methods, function and classes etc.

They are executed only the first time the module name is encountered in an import statement.

The file name is the module name with the suffix .py appended.
Ex:- mymodule.py

Type of Modules:-

- User-defined Modules
- Built-in Modules

Built-in Modules:

One of the many superpowers of Python is that it comes with a "rich standard library".

This rich standard library contains lots of built-in modules.
Hence, it provides a lot of reusable code.

To name a few, Python contains modules like
Ex:- array, math, numpy, sys ,os, sys, datetime, random.

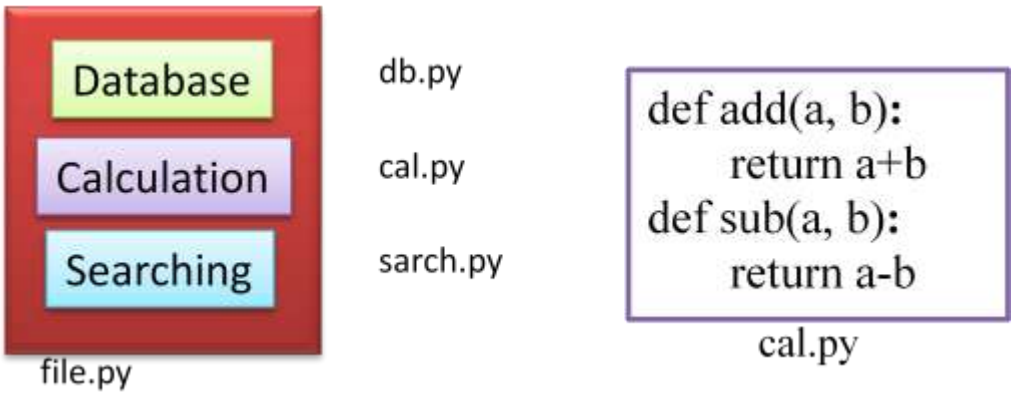
When and Why use Module

Assume that you are building a very large project, it will be very difficult to manage all logic within one single file so If you want to separate your similar logic to a separate file, you can use module.

It will not only separate your logics but also help you to debug your code easily as you know which logic is defined in which module.

When a module is developed, it can be reused in any program that needs that module.

Creating a Module



import statement is used to import modules.

Syntax:-

```
1.Import module_name class_name1,class_name2,……, class_nameN
2.import module_name as alias_name
3.from module_name import
4.from module_name import f_name as alias_f_name
5.from module_name import *
```

1.import module_name

This does not enter the names of the functions defined in module directly in the current symbol table; it only enters the module name there.
Ex:- import cal

How to access Methods, Functions, Variable and Classes ?

Using the module name you can access the functions.

Syntax:- module_name.function_name()

Ex:-

```
cal.add(10, 20)
cal.sub(20, 10)
add = cal.add
add(10, 20)
```

When 2 modules having same function name then
This import module is good approach to use.

2.import module_name as alias_name

This does not enter the names of the functions defined in module directly in the current symbol table; it only enters the module name there.

If the module name is followed by *as*, then the name following as is bound directly to the imported module.

Ex:- import cal as c

How to access Methods, Functions, Variable and Classes ?

Using the alias_name you can access the functions.

Ex:-

```
c.add(10, 20)
c.sub(20, 10)
add = c.add
add(10, 20)
```

3.from module_name import function_name

There is a variant of the import statement that imports names from a module directly into the importing module's symbol table.

Ex:- from cal import add, sub

How to access Methods, Functions, Variable and Classes ?

You can access the functions directly by it's name.

Ex:-

```
add(10, 20)
sub(20, 10)
```

4.from module_name import f_name as a_name

Ex:- from cal import add as s

How to access Methods, Functions, Variable and Classes ?

You can access the functions directly by it's alias name.

Ex:-

```
s(10, 20)
```

5.from module_name import *

imports all names except those beginning with an underscore (_).

Ex:- from cal import *

How to access Methods, Functions, Variable and Classes ?

You can access the functions directly by it's name.

Ex:-

```
add(10, 20)
sub(20, 10)
```

```python
#cal.py  <--- cal Module
a = 50

def name():
    print("From Module cal")

def add(a,b):
    return a+b

def sub(a,b):
    return a-b
```

```python
# Demo.py <--- Main Module

import cal              # Importing Cal Module

print("cal Module's variable:", cal.a)  # Accessing Cal Module's Variable

cal.name()              # Accessing Cal Module's Function

a = cal.add(10,20)      # Accessing Cal Module's Function
print(a)

b = cal.sub(20, 10)     # Accessing Cal Module's Function
print(b)
```

```
import cal              # Importing Cal Module
import cal as c         #replace cal by c
from   cal import a, name, add, sub #no need to use cal.
from   cal import a, name, add as s, sub
from   cal import *
```

Abstract Class

A class derived from *ABC* class which belongs to *abc* module, is known as abstract class in Python.

ABC Class is known as Meta Class which means a class that defines the behavior of other classes. So we can say, Meta Class ABC defines that the class which is derived from it becomes an abstract class.

Abstract Class can have abstract method and concrete methods.

Abstract Class needs to be extended and its method implemented.

PVM can not create objects of an abstract class.

Ex:-
```
from abc import ABC, abstractmethod
Class Father(ABC):
```

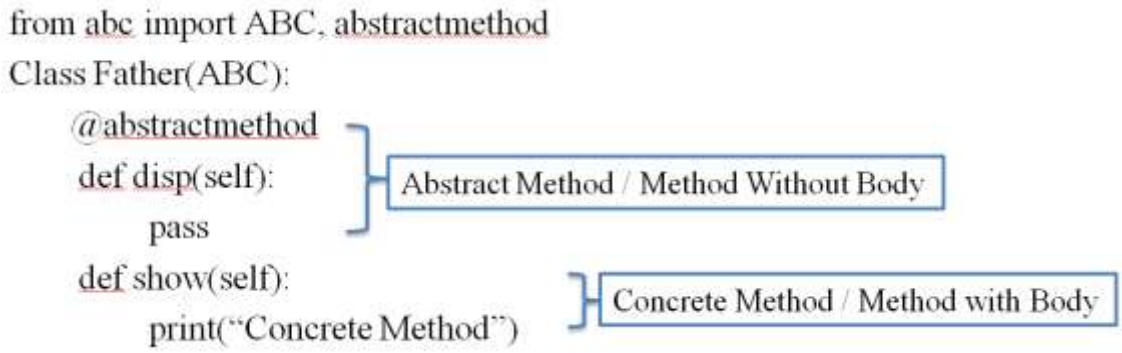
Abstract Method

A abstract method is a method whose action is redefined in the child classes as per the requirement of the object.
We can declare a method as abstract method by using @abstractmethod decorator.

Ex:-
```
from abc import ABC, abstractmethod
Class Father(ABC):
        @abstractmethod
         def disp(self):
                 pass
```

Concrete Method

A Concrete method is a method whose action is defined in the abstract class itself.
Ex:-



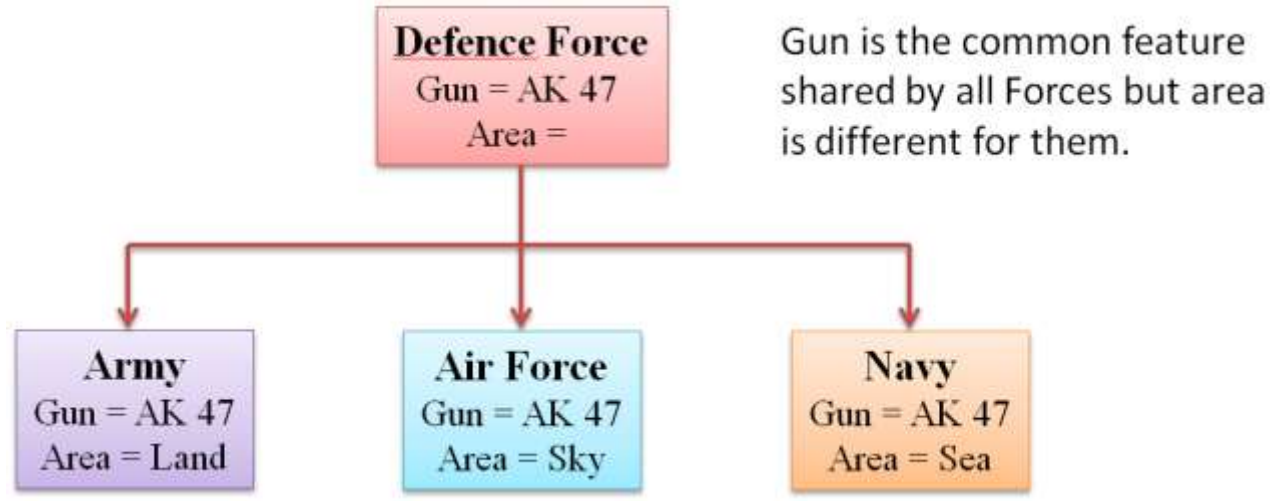
Rules:

- PVM can not create objects of an abstract class.
- It is not necessary to declare all methods abstract in a abstract class.
- Abstract Class can have abstract method and concrete methods.
- If there is any abstract method in a class, that class must be abstract.
- The abstract methods of an abstract class must be defined in its child class/subclass.
- If you are inheriting any abstract class that have abstract method, you must either provide the implementation of the method or make this class abstract.

```
from abc import ABC, abstractmethod
class Father(ABC):

     @abstractmethod
     def disp(self):          # Abstract Method
          pass

     def show(self):          # Concrete Method
          print('Concrete Method')

#my = Father()  #Not possible to create object of a abstract class

class Child(Father):
     def disp(self):
          print("Defining Abstract Method")


c = Child()
c.disp()
c.show()
```

When use Abstract Class

We use abstract class when there are some common feature shared by all the objects as they are.



```
from abc import ABC, abstractmethod

class DefenceForce (ABC):
     def __init__(self):
          self.id = 101

     @abstractmethod
     def area(self):
          pass

     def gun(self):
          print("Gun = AK47")

class Army(DefenceForce):
     def area(self):
          print("Army Area = Land", self.id)

class AirForce(DefenceForce):
     def area(self):
          print("AirForce Area = Sky", self.id)

class Navy(DefenceForce):
     def area(self):
          print("Navy Area = Sea", self.id)
```

```
a = Army()
af = AirForce()
n = Navy()

a.gun()
a.area()
print()
af.gun()
af.area()
print()
n.gun()
n.area()
```

output:

```
Gun = AK47
Army Area = Land 101

Gun = AK47
AirForce Area = Sky 101

Gun = AK47
Navy Area = Sea 101
```

Interface

In Python, The interface concept is not explicitly available, like available in other languages e.g. Java.

In Python, an interface is an abstract class which contains only abstract method but not a single concrete method.

```
from abc import ABC, abstractmethod
class Father(ABC):
     @abstractmethod
     def disp(self):
          pass
     def show(self):
          print("Concrete Method")
```

Rules

All methods of an interface is abstract.

We can not create object of interface.

If a class is implementing an interface it has to define all the methods given in that interface.

If a class does not implement all the methods declared in the interface, the class must be declared abstract.

```
from abc import ABC, abstractmethod
class Father(ABC):

     @abstractmethod
     def disp(self):      # Abstract Method
          pass

#my = Father()  #Not possible to create object of a abstract class

class Child(Father):
     def disp(self):
          print("Child Class")
          print("Defining Abstract Method")


c = Child()
c.disp()
```

When use Interface

We use interface when all the features need to be implemented differently for different objects.


```
from abc import ABC, abstractmethod

class Father(ABC):
     @abstractmethod
     def disp1(self):
          pass
     @abstractmethod
     def disp2(self):
          pass

class Child(Father):
     def disp1(self):
          print("Child Class")
          print("Disp 1 Abstract Method")

class Granchild(Child):
     def disp2(self):
          print("GrandChild Class")
          print("Disp 2 Abstract Method")

gc = Granchild()
gc.disp1()
gc.disp2()
```

```
Output:
Child Class
Disp 1 Abstract Method
GrandChild Class
Disp 2 Abstract Method
```

Abstract Class vs Interface

- An abstract class can have abstract methods as well as concrete methods, but All methods of an interface are abstract.
- We use abstract class when there are some common feature shared by all the objects as they are while we use interface if all the feature need to be implemented differently for different objects.
- Its programmer job to write child class for abstract class while in interface, any third party vendor will take responsibility to write child class.
- Interfaces are slow when compared to abstract class.

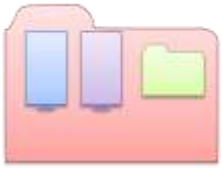
Package

Packages are a way of structuring Python's module namespace by using "dotted module names".

A package can have one or more modules which means, a package is collection of modules and packages.

A package can contain packages.

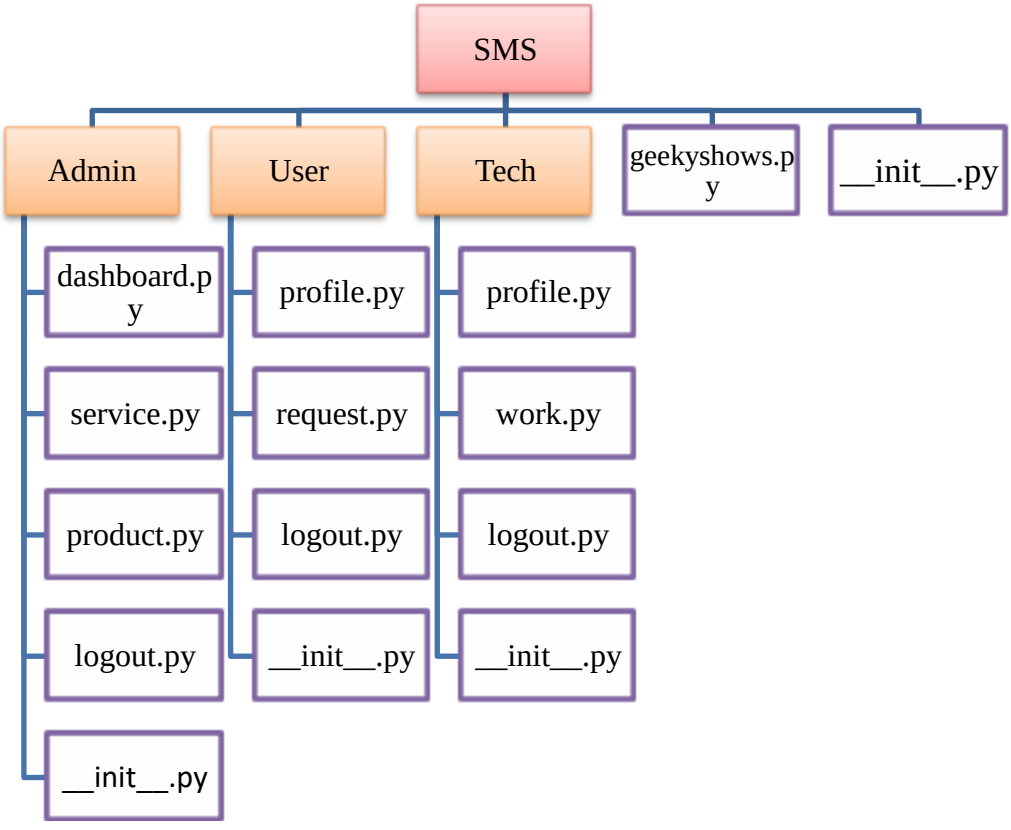
Package is nothing but a Directory/Folder

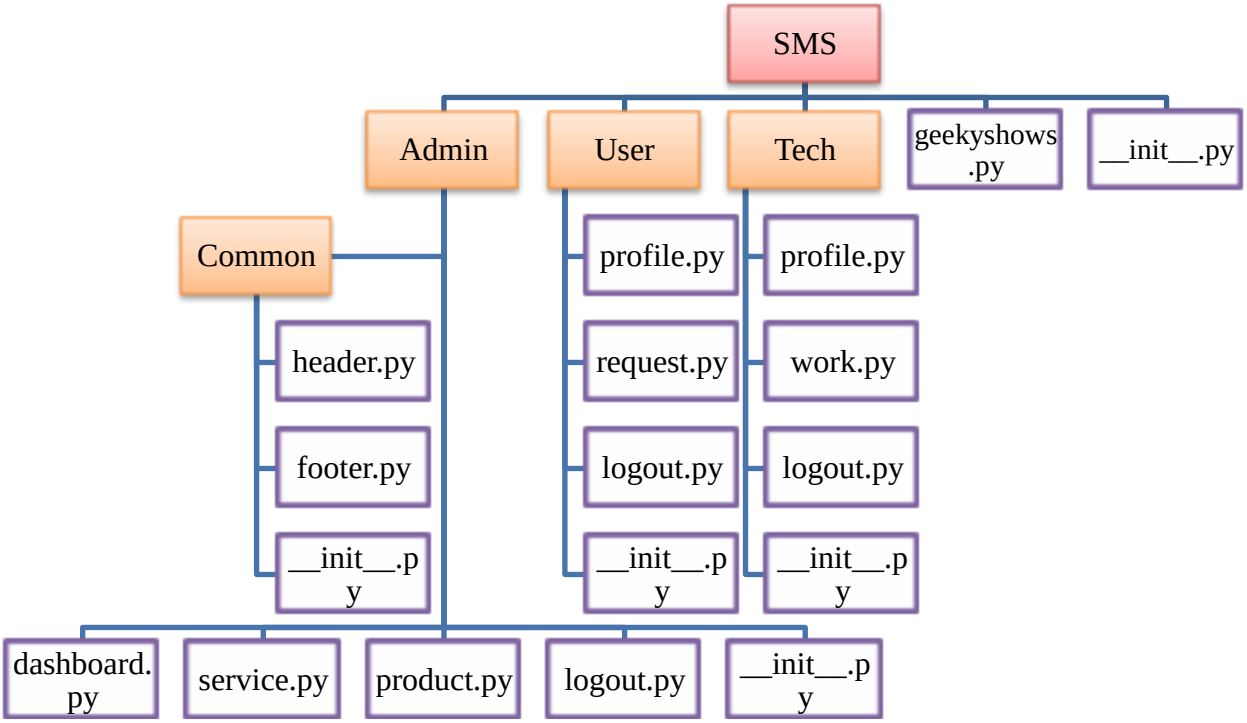


Creating Package

Package is nothing but a Directory/Folder which MUST contain a special file called ***__init__.py***.

__init__.py file can be empty, it indicates that the directory it contains is a Python package, so it can be imported the same way a module can be imported.





How to use Package

```
Syntax:- import packageName.moduleName
Syntax:- import packageName.subPackageName.moduleName

Ex:- import Admin.service
Ex:- import Admin.Common.footer
```

How to Access Variable, Function, Method, Class etc. ?

```
Syntax:- packageName.moduleName.functionName()
Syntax:- packageName.subPackageName.moduleName.functionName()

Ex:- Admin.service.admin_service( )
Ex:- Admin.Common.footer.admin_common_footer( )
```

```
Syntax:- from packageName import moduleName
Syntax:- from packageName.subPackageName import moduleName

Ex:- from Admin import service
Ex:- from Admin.Common import footer
```

How to Access Variable, Function, Method, Class etc.

```
Syntax:- moduleName.functionName()

Ex:-
service.admin_service( )
footer.admin_common_footer( )
```

Syntax:- from packageName import *
Syntax:- from packageName.subPackageName import *
Ex:- from Admin import *
Ex:- from Admin.Common import *

How to Access Variable, Function, Method, Class etc.

Syntax:- moduleName.functionName()
Ex:-
service.admin_service( )
footer.admin_common_footer( )

Exception

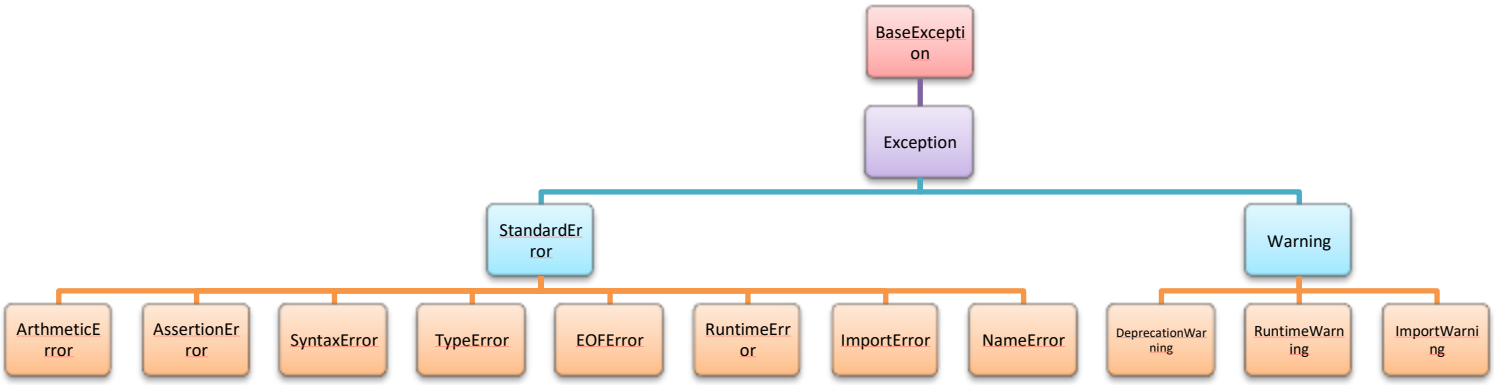
An exception is a runtime error which can be handled by the programmer.
All exceptions are represented as classes in Python.

Type of Exception:-

1.Built-in Exception –
Exceptions which are already available in Python Language. The base class for all built-in exceptions is BaseException class.

2.User Defined Exception –
A programmer can create his own exceptions, called user-defined exceptions.

All exceptions are represented as classes in Python.



Need of Exception Handling

- When an exception occurs, the program terminates suddenly.
- Suddenly termination of program may corrupt the program.
- Exception may cause data loss from the database or a file.

Try – The try block contains code which may cause exceptions.
Syntax-

```
try:
    statements
```

Except – The except block is used to catch an exception that is raised in the try block. There can be multiple except block for try block.
Syntax-

```
except ExceptionName:
      statements
```

Else – This block will get executed when no exception is raised. Else block is executed after try block.
Syntax-

```
else:
      statements
```

Finally – This block will get executed irrespective of whether there is an exception or not.
Syntax-

```
finally:
      statements
```

- We can write several except blocks for a single try block.
- We can write multiple except blocks to handle multiple exceptions.
- We can write try block without any except blocks.
- We can not write except block without a try block.
- Finally block is always executed irrespective of whether there is an exception or not.
- Else block is optional.
- Finally block is optional.

```
try:
    Statement
except ExceptionClassName:
    Statement
else:
   Statement
finally:
    Statement

try:
    Statement
except ExceptionClassName:
    Statement
```

```
try:
    Statement
except ExceptionClassName1:
    Statement
except ExceptionClassName2:
    Statement
finally:
    Statement

try:
    Statement

except ExceptionClassName:
    Statement
```

```
#a = 10
#b = 0
#d = a/b

#print(d)
#print('Rest of the Code')
```

1.tryExcept

```
a = 10
b = 0
try:
    d = a/b
    print(d)
    print('Inside Try')

except ZeroDivisionError:
    print('Division by Zero Not allowed')

print('Rest of the Code')
```

2.tryExceptElse

```
a = 10
b = 5
try:
    d = a/b
    print(d)
    print('Inside Try')

except ZeroDivisionError:
    print('Division by Zero Not allowed')

else:
    print('Inside Else')

print('Rest of the Code')
```

3.tryExceptelse Finally

```
a = 10
b = 0
try:
    d = a/b
    print(d)
    print('Inside Try')

except ZeroDivisionError:
    print('Division by Zero Not allowed')

else:
    print('Inside Else')

finally:
    print('Inside Finally')

print('Rest of the Code')
```

Except

```
With the Exception Class Name
            except ExceptionClassName:
                    Statement

Exception as an object
            except ExceptionClassName as obj:
                    Statement

Multiple Exception within tuple
            except (ExceptionClassName1,
            ExceptionClassName2, ExceptionClassName3, ……
            ):
                    Statement

Catch any Type of Exception
            except:
                Statement
```

```python
a = 10
b = 0
try:
    d = a/b
    print(d)
    print('Inside Try')

except ZeroDivisionError as obj:
    print(obj)

print('Rest of the Code')
```

```python
a = 10
b = 0

try:
    d = a/b
    print(d)

except ZeroDivisionError as obj:
    print(obj)

except NameError as ob:
    print(ob)

print('Rest of the Code')
```

```python
a = 10
b = 0
try:
    d = a/b
    print(d)

except (NameError, ZeroDivisionError) as obj:
    print(obj)

print('Rest of the Code')
```

```python
a = 10
b = 0
try:
    d = a/b
    print(d)

except:
    print('Exception Handler')

print('Rest of the Code')
```

Assert Statement

The assert Statement is useful to ensure that a given condition is True.
If it is not true, it raises AssertionError.

Syntax:- assert condition, error_message

- If the condition is False then the exception by the name AssertionError is raised along with the message.
- If message is not given and the condition is False then also AssertionError is raised without message.

User Defined Exception

A programmer can create his own exceptions, called user-defined exceptions or Custom Exception.

- Creating Exception Class using Exception Class as a Base Class
- Raising Exception
- Handling Exception

Creating Exception

We can create our own exception by creating a sub class to built-in Exception class.

```
class MyException(Exception):
        pass
class MyException(Exception):
        def __init__(self, arg):
    self.msg = arg
```

Raising Exception

raise statement is used to raise the user defined exception.

```
raise MyException('message')
```

```
a = 20
assert a <= 10, 'Invalid Number'
```

```
class BalanceException (Exception):
    #pass
    def __init__(self, arg):
        self.msg = arg

def checkbalance():
    money = 10000
    withdraw = 9000
    try:
        balance = money - withdraw
        if(balance<=2000):
            raise BalanceException('Insufficient Balance')
        print(balance)
    except BalanceException as be:
        print(be)

checkbalance()  #Insufficient Balance
```

Error vs Exception

- An exception is an error that can be handled by a programmer.
- An exception which are not handled by programmer, becomes an error.
- All exceptions occur only at runtime.
- Error may occur at compile time or runtime.

Error vs Warning

It is compulsory to handle all error otherwise program will not execute, while warning represents a caution and even though it is not handled, the program will execute.

Errors are derived as sub class of *StandardError*, while warning derived as sub class from *Warning* class.